Intelligenza Artificiale

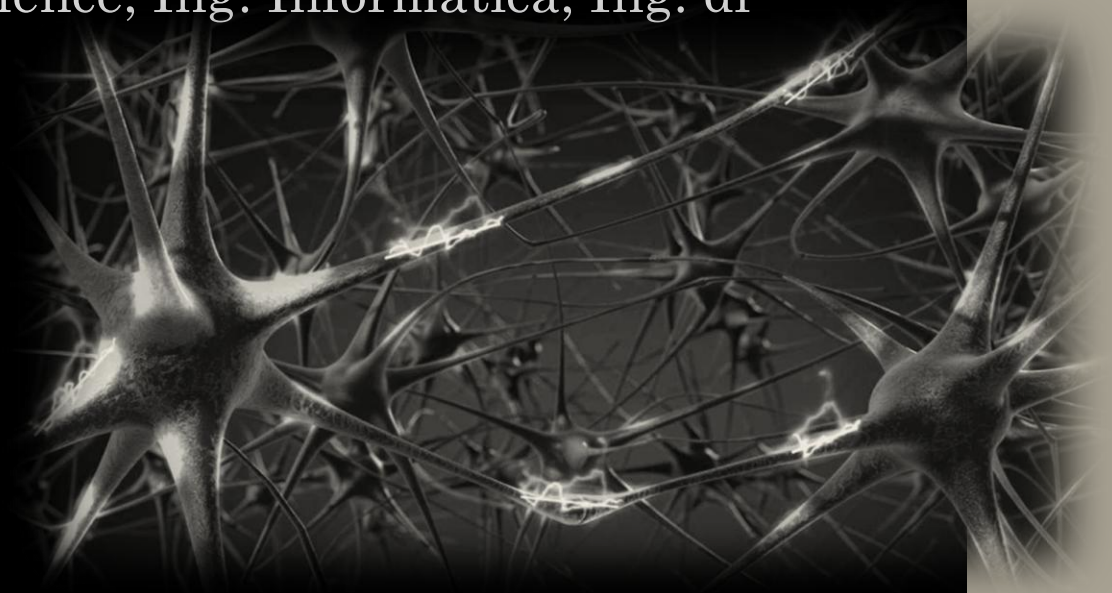
Apprendimento Automatico da esempi

Corsi di Laurea in Informatica, Ing. Gestionale, Metodi Matemaici per la Data Science, Ing. Informatica, Ing. di Internet

(a.a. 2025-2026)

Roberto Basili

(*) dalle *slides* di S. Russel



Overview (AIMA chpt. 18.1-18.4)

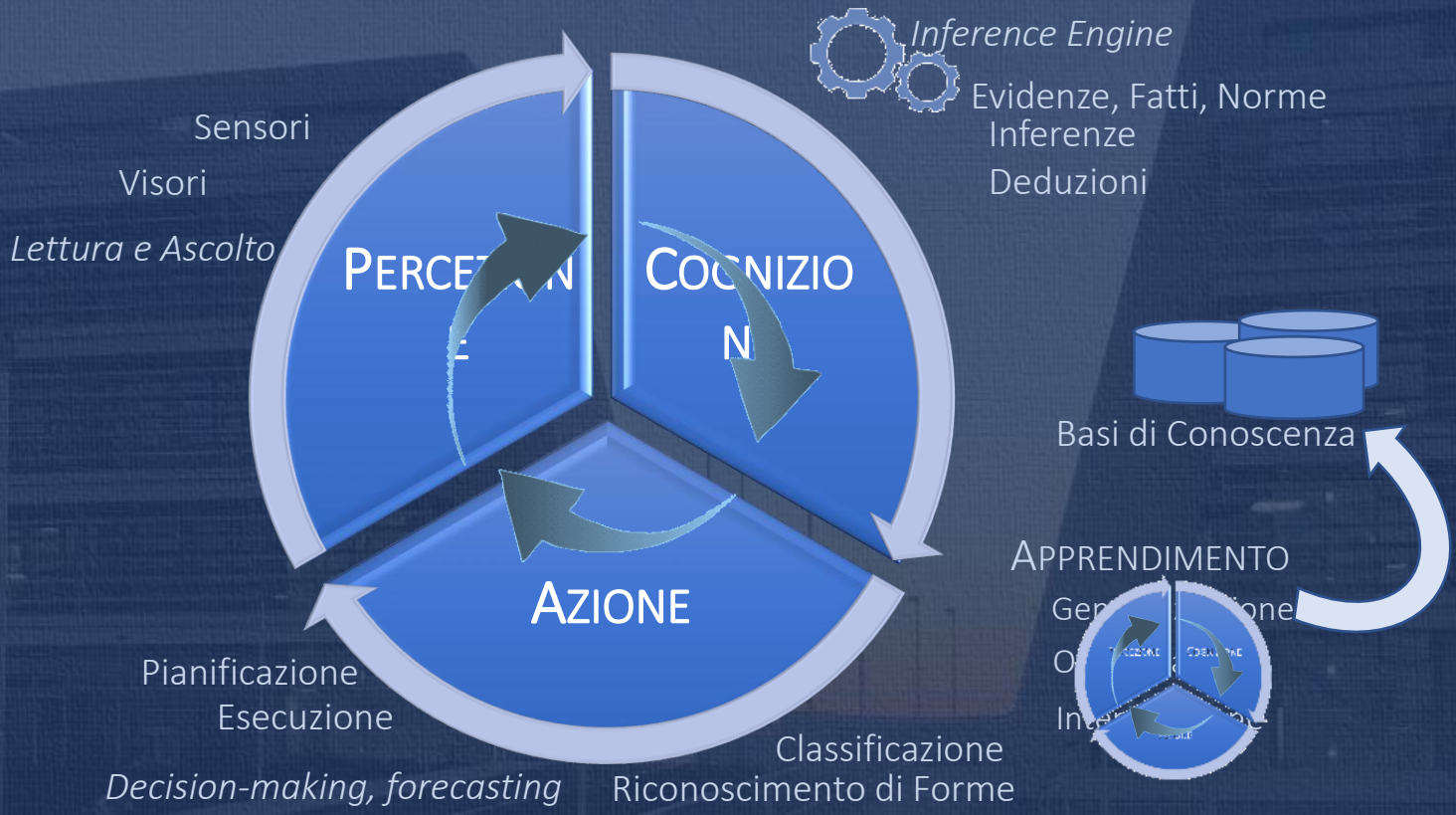
- Agents & machine learning
- Learning from examples:
  - Complexity and Expressiveness
  - The definition of model selection
- Performance Evaluation
- Learning methodology: design, experiment/ evaluation and model selection
  - Cross validation
- An example: Decision Tree learning
  - Recursive search among Boolean formulas
  - Attribute Selection in DT: Information Gain

Introduzione al Machine Learning

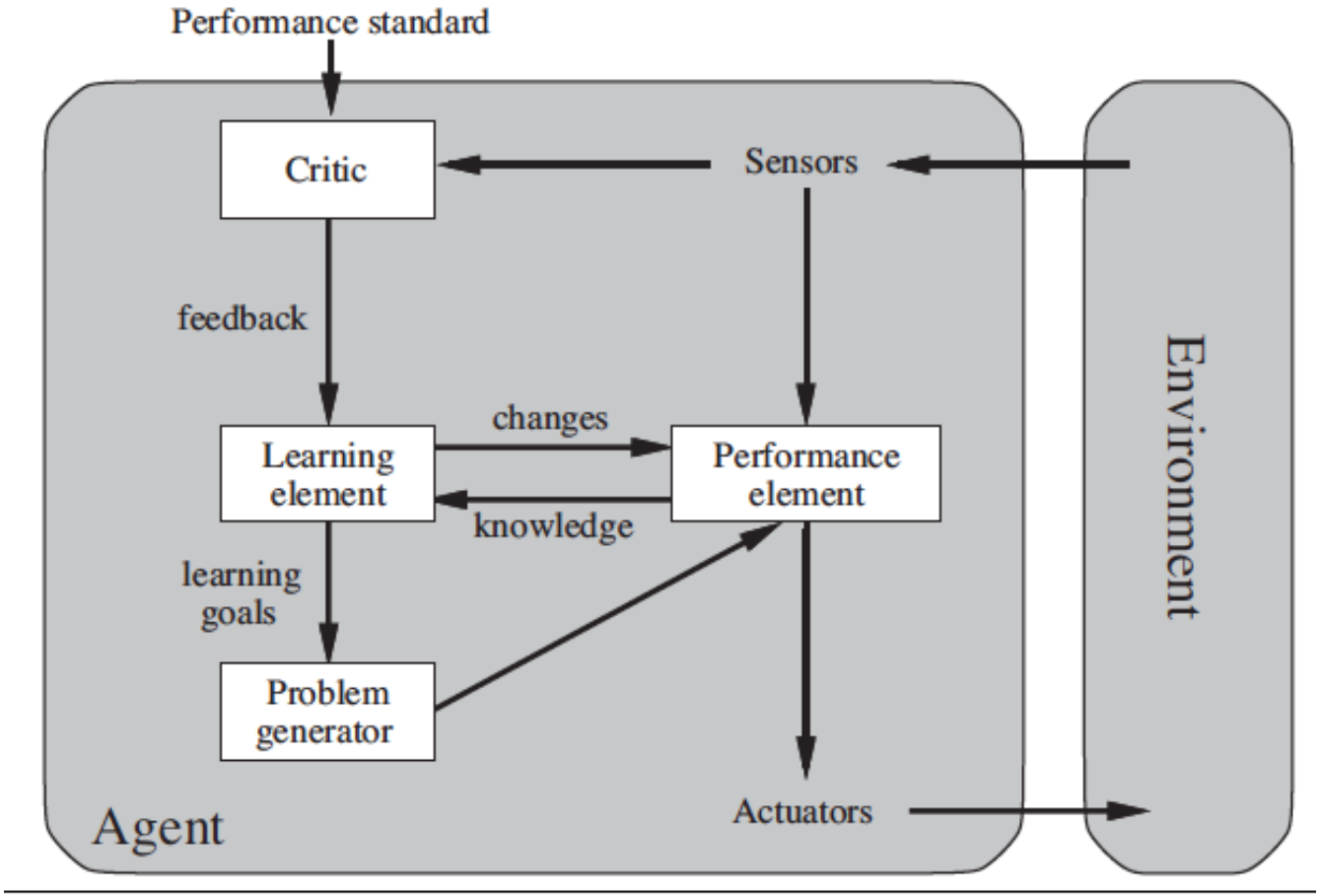
- Introduzione al ML
  - Cos'è il ML:
    - Qual'è l'obbiettivo
    - Come si applica
  - Metodologia del Learning: ML design, experiment, ML evaluation
  - Aspetti del ML: quale rappresentazione delle ipotesi?

  - Paradigmi di Machine Learning
    - Supervised learning, Apprendimento per esempi
    - Unsupervised learning, apprendimento senza supervisione
    - Reinforcement learning, apprendimento per rinforzo

Agente AI e Apprendimento Automatico



AIMA learning architecture



Machine learning: definition

- *A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E* [Mitchell, 1997]
- Definizione del problema per un *learning agent*
  - Task T
  - Performance measure P
  - Experience E

Designing a learning system

1. Choosing and representing the *training experience* (as instances of world states)
   - Examples of best moves, games outcome …
2. Choosing a *target decision function, h*
   - board-move, board-value, …
3. Choosing a *representation* for the target function, *h*
   - e.g., linear function with weights (hypothesis space)
4. Choosing a *learning algorithm* for approximating the target function
   - A method for parameter estimation

Inductive learning

- Simplest form: learn a function from examples
    - f is the target decision function
    - An example is a pair $(x, f(x))$

Problem: find a hypothesis h

such that $h \approx f$

given a training set of examples

(This is a highly simplified model of real learning:

- Ignores prior knowledge
- Assumes examples are given)

Inductive learning: an example

Simplest form: learn a function from examples

- *f* is the target decision function, e.g. which move in a labyrinth
- An example is a pair (*x*, *f(x)*), e.g. the state description *x* and the proper move *f(x)* in *x*

Problem:

Find a hypothesis *h(x)* e.g. decision *h(x)* about the move in x
such that $h(x) \approx f(x)$
given the training set of example pairs *(x,f(x))*

(This is a highly simplified model of real learning:
- Ignores prior knowledge
- Assumes examples are given)

Inductive learning: a further example

Simplest form: learn a function from examples

- *f* is the target decision function, e.g. which sentiment in a tweet?
- An example is a pair (*x*, *f(x)*),
  e.g. the tweet description *x* and its sentiment class *f(x)*

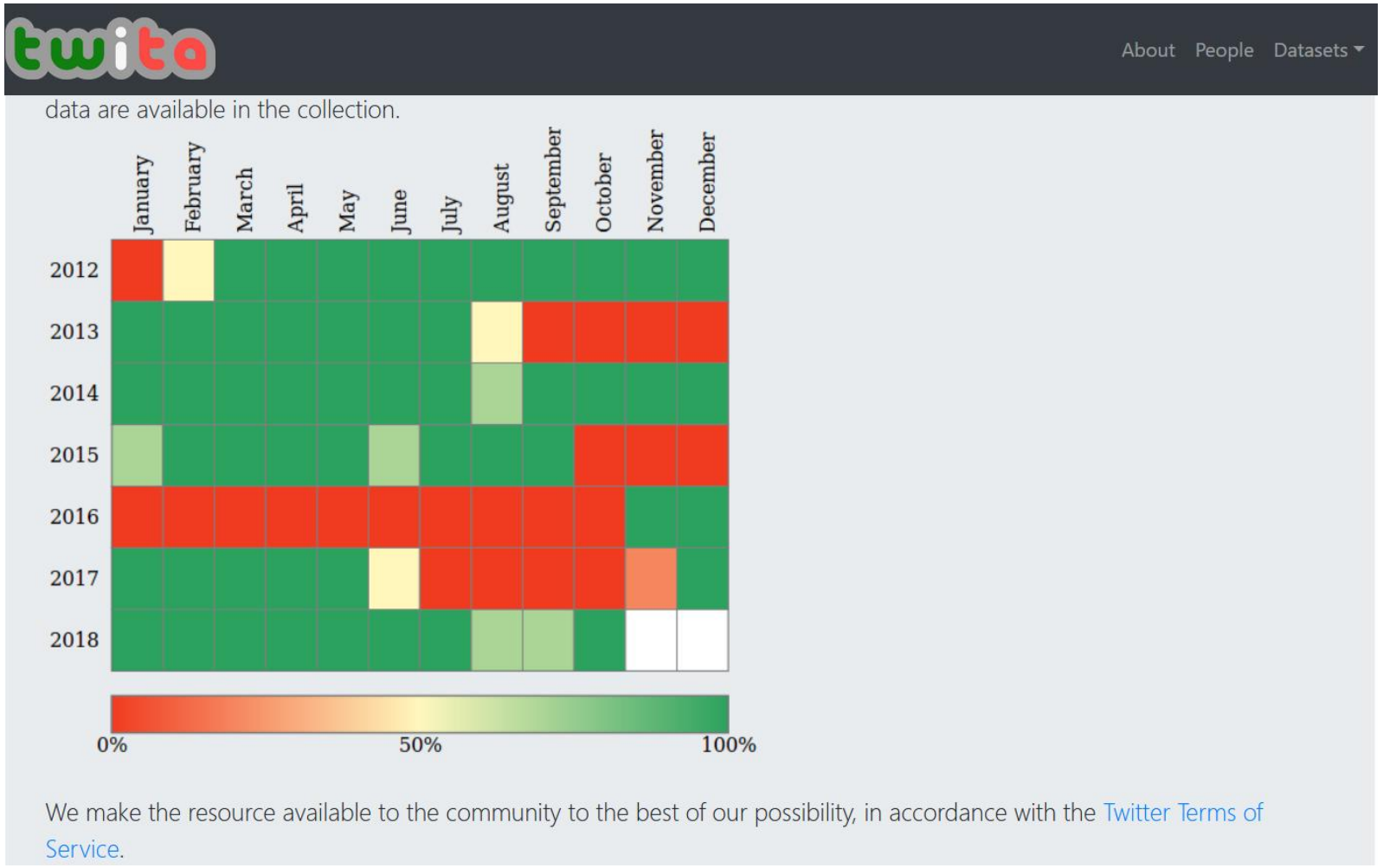
Problem:

Find a hypothesis *h(x)* e.g. decision *h(x)* about the sentiment of x
such that $h(x) \approx f(x)$
given the training set of example pairs *(x,f(x))*

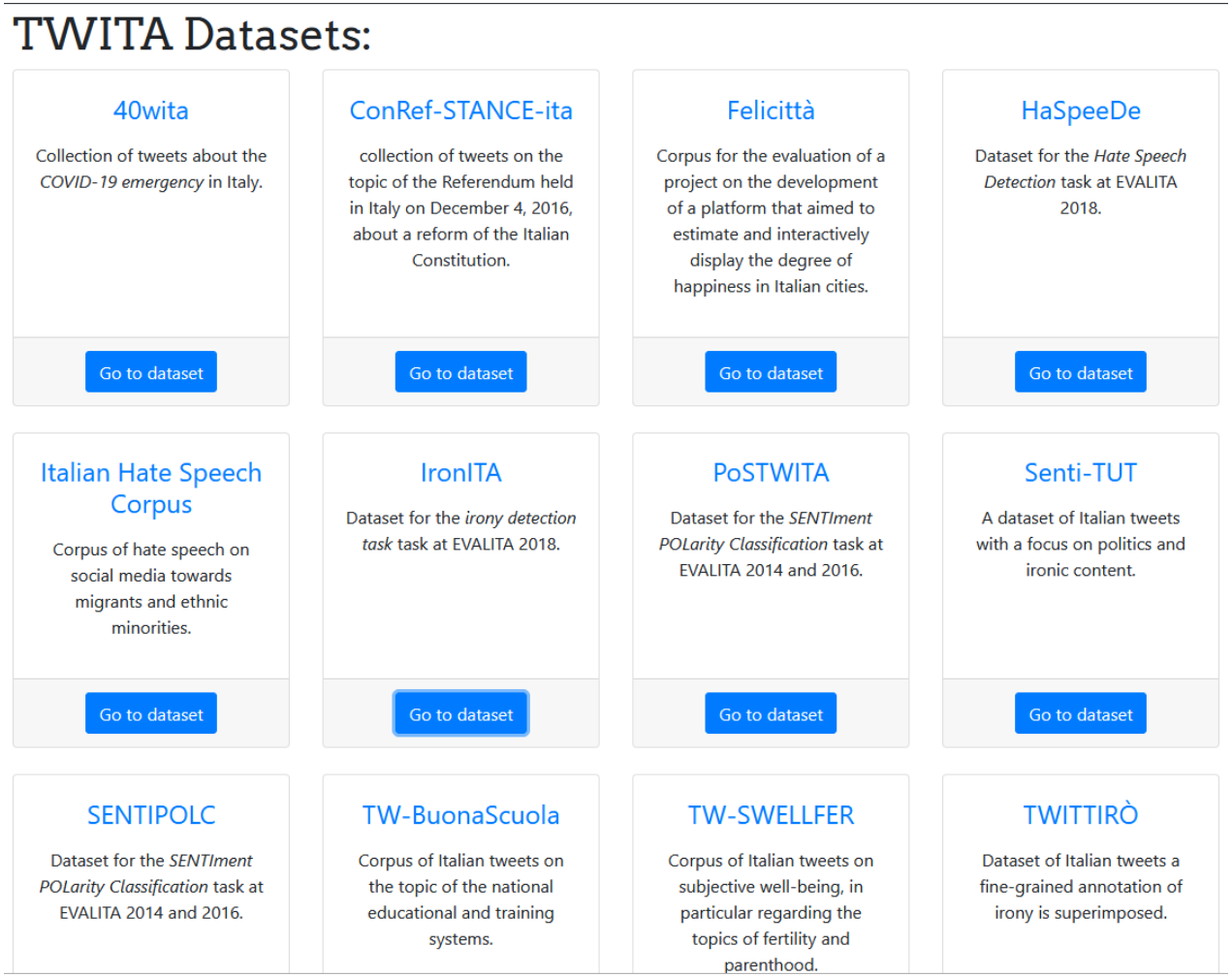
(This is a highly simplified model of real learning:

- Ignores prior knowledge: *which are the topics under discussion?*
- Assumes examples are given: *How to gather labeled tweets?*

Labeled Corpora on Twitter: Twita (https://twita.di.unito.it/)



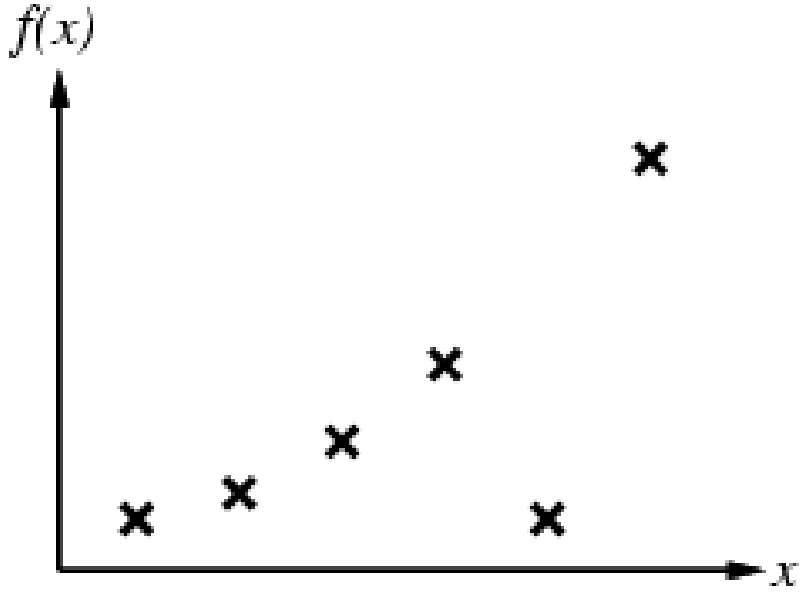
Twita datasets:



Inductive learning method

- Construct/adjust *h* to agree with *f* on training set
  - *h* is consistent if it agrees with *f* on all examples), that is h(x)=f(x) for all x in the training dataset

e.g., *curve fitting*:

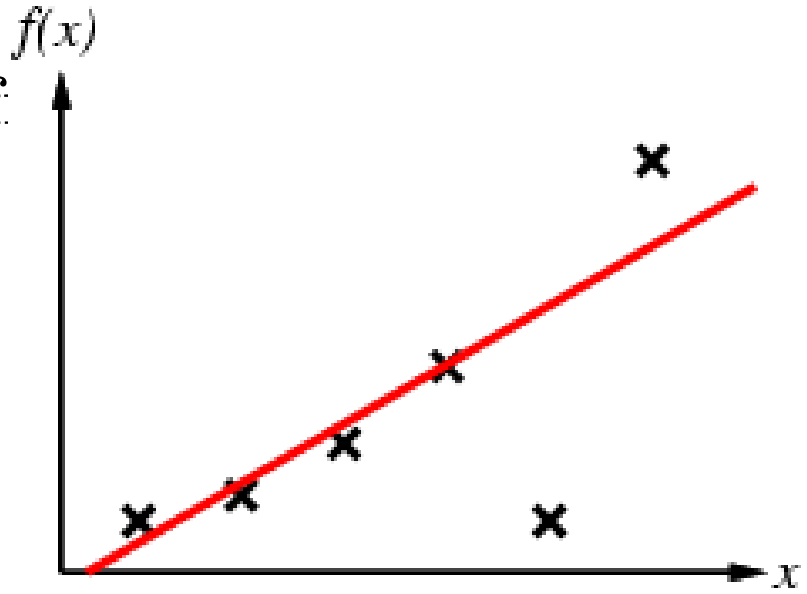


Inductive learning method

- Construct/adjust *h* to agree with *f* on training set

(*h* is consistent if it agrees with *f* on all examples)

e.g., curve f

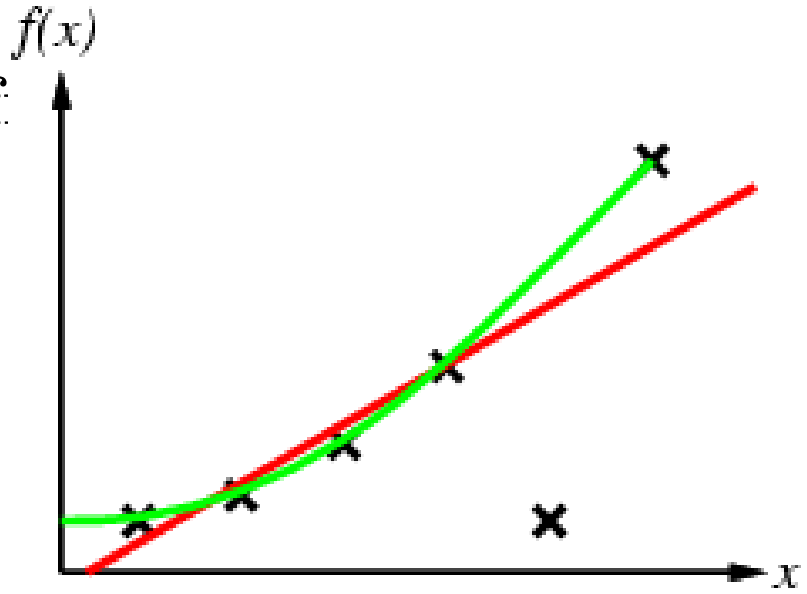


Inductive learning method

- Construct/adjust *h* to agree with *f* on training set

(*h* is consistent if it agrees with *f* on all examples)

e.g., curve f

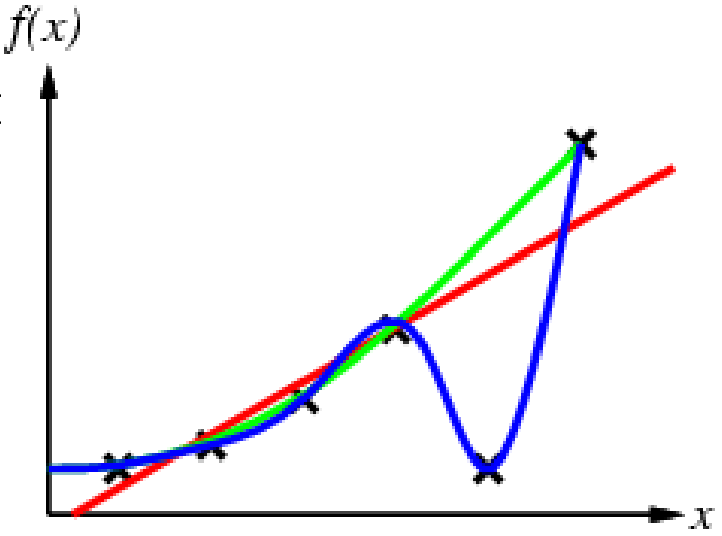


Inductive learning method

- Construct/adjust *h* to agree with *f* on training set

(*h* is consistent if it agrees with *f* on all examples)

e.g., curve f

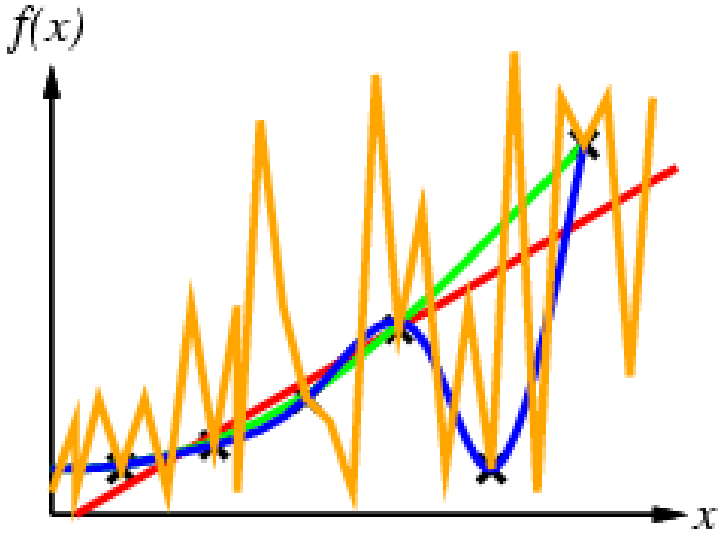


Inductive learning method

- Construct/adjust h to agree with f on training set

(h is consistent if it agrees with f on all examples)

e.g., curve fitting:

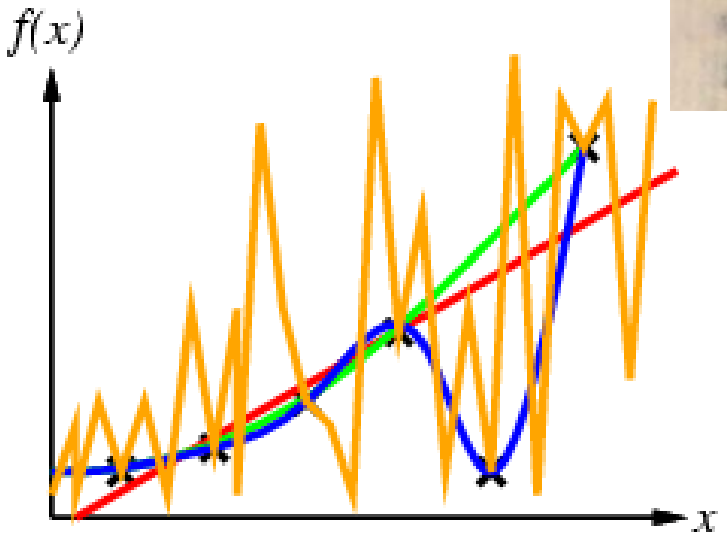


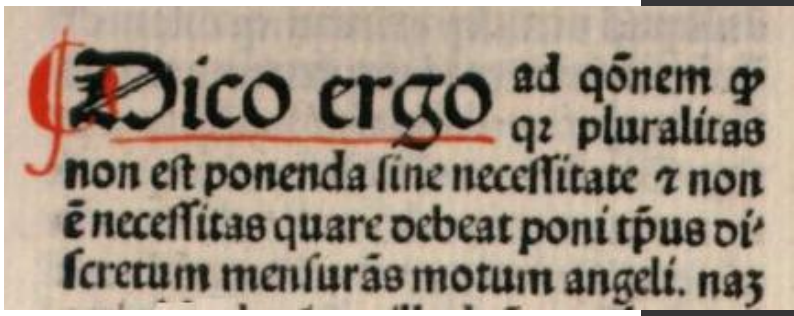
Inductive learning method

- Construct/adjust *h* to agree with *f* on training set

(*h* is consistent if it agrees with *f* on all examples)

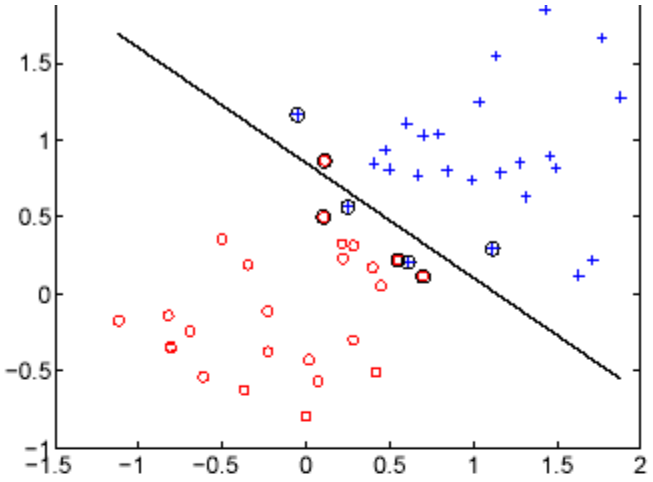
E.g., curve fitting:



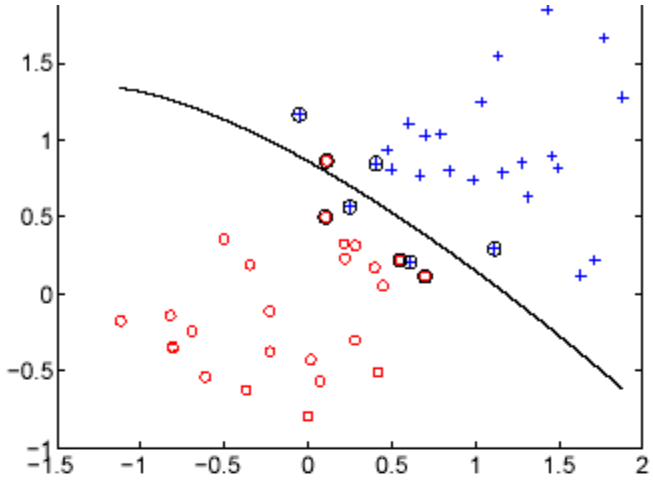


novacula Occami
Entia non sunt multiplicanda praeter necessitatem

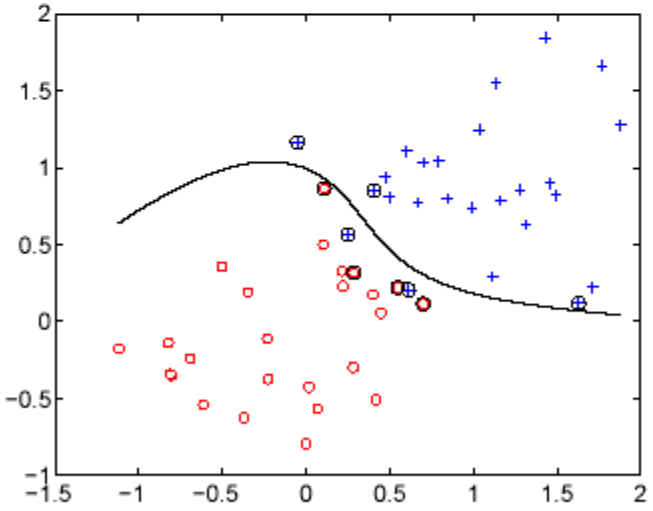
Ockham's razor:
prefer the simplest hypothesis consistent with data



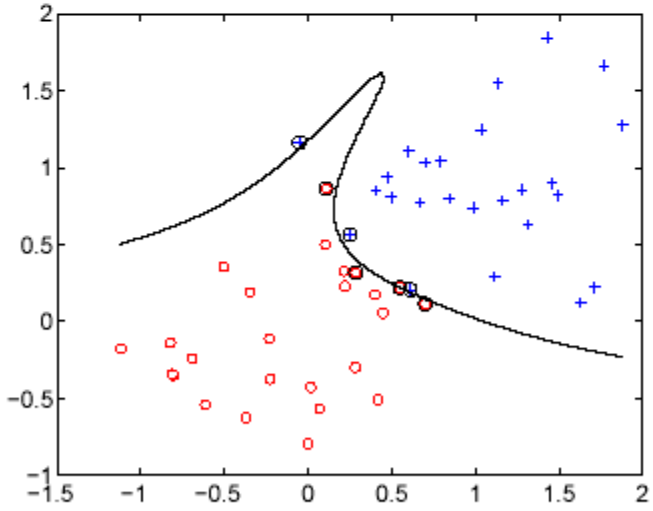
linear



2nd order polynomial



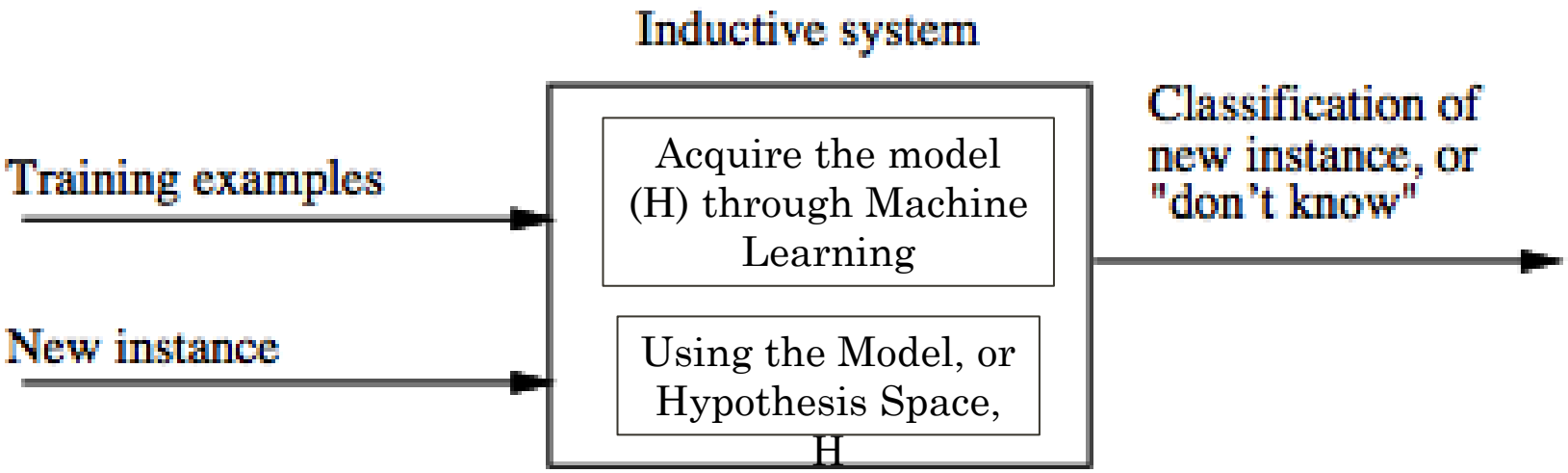
4th order polynomial



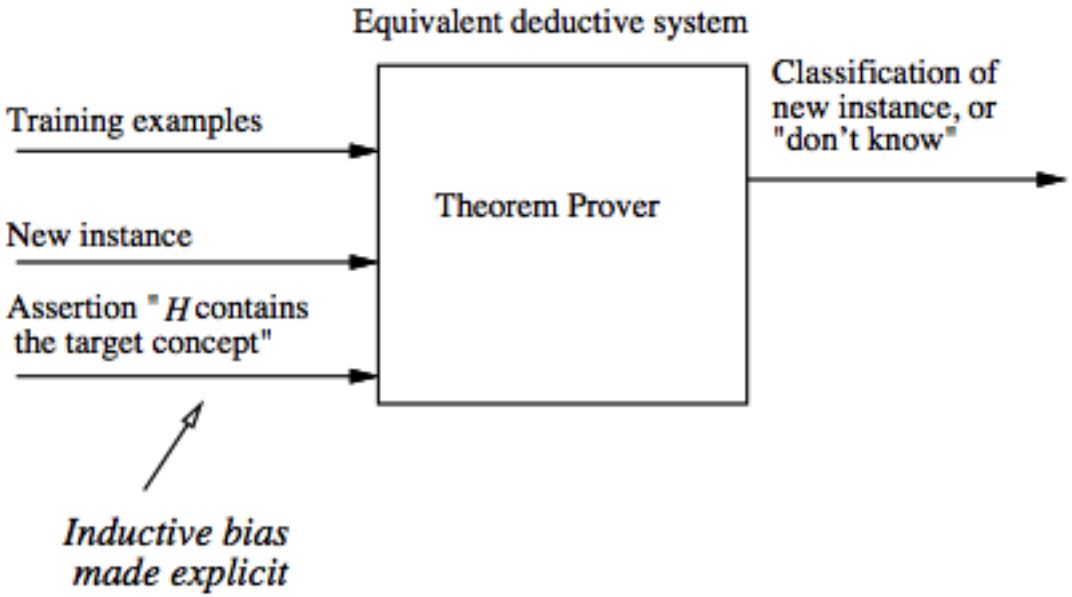
8th order polynomial

Algoritmi di ML o *Learning Machines*

Inductive system



Equivalent deductive system



Learning decision trees

Problem: decide whether to wait for a table at a restaurant, based on the following attributes:

1. Alternate: is there an alternative restaurant nearby?
2. Bar: is there a comfortable bar area to wait in?
3. Fri/Sat: is today Friday or Saturday?
4. Hungry: are we hungry?
5. Patrons: number of people in the restaurant (None, Some, Full)
6. Price: price range ($, $$, $$$)
7. Raining: is it raining outside?
8. Reservation: have we made a reservation?
9. Type: kind of restaurant (French, Italian, Thai, Burger)
10. WaitEstimate: estimated waiting time (0-10, 10-30, 30-60, >60)

Attribute-based representations

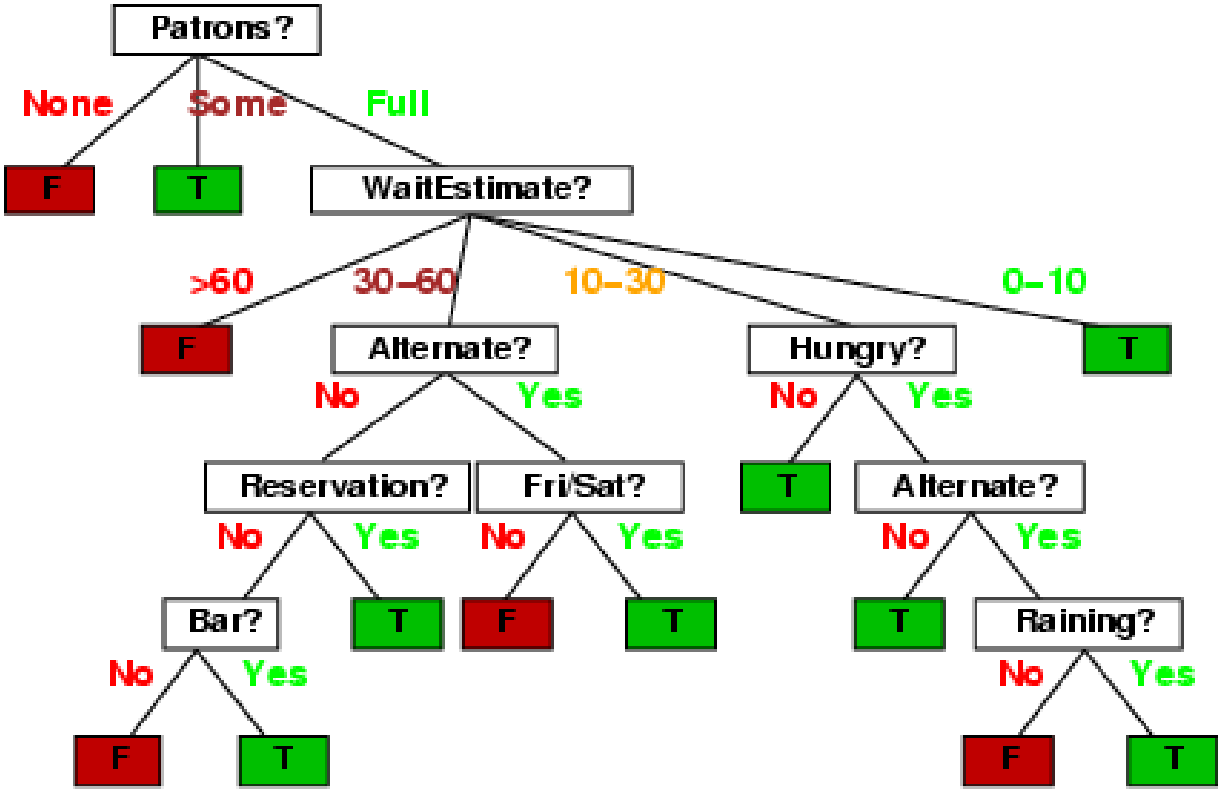
- Examples described by attribute values (Boolean, discrete, continuous)
- E.g., situations where I will/won't wait for a table:

Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*Wait*
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T
X_9	F	T	T	F	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

- Classification of examples is positive (T) or negative (F)

Decision trees

- One possible representation for hypotheses
- E.g., here is the "true" tree for deciding whether to wait:

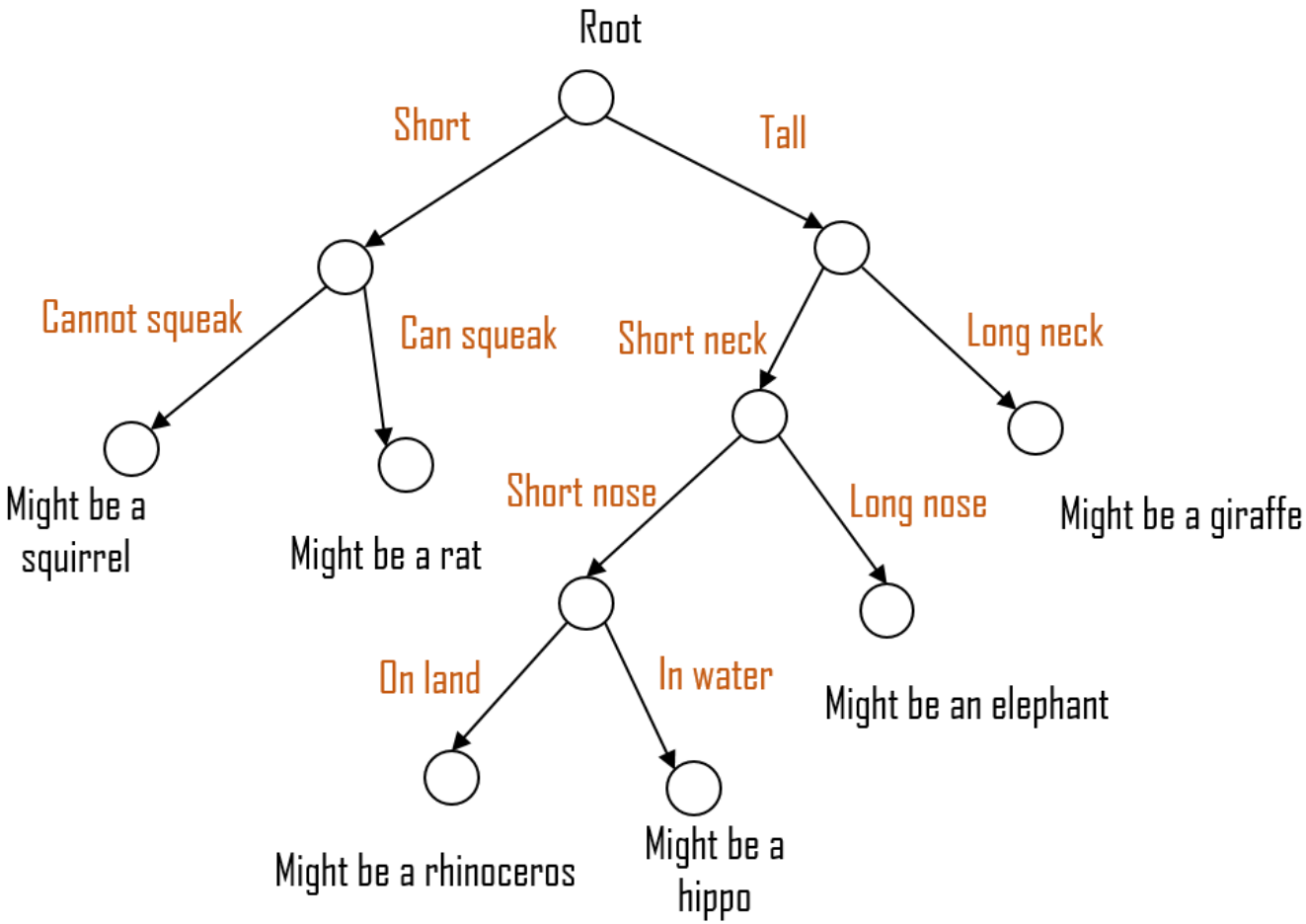


Use of a decision tree

- In a **decision tree**:
  - An inner node specifies a test of some attribute of an instance;
  - A branch descending from a node corresponds to one of the possible values the attribute can assume;
  - A leaf node assigns a classification.
- The classification of an instance corresponds to:
  1. START from the root;
  2. SELECT the attribute attached to the current node;
  3. MOVE DOWN the tree branch corresponding to the value of that attribute in the given instance;
  4. IF a leaf node is reached, the label associated to the node is RETURNED,

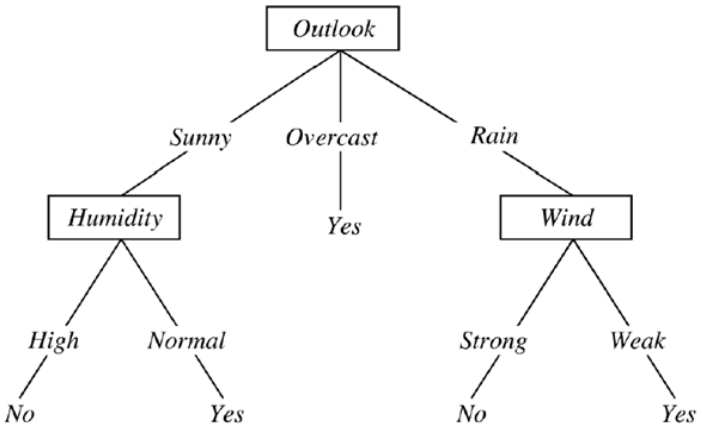
     ELSE go to step 2 and repeat.

Another example



DTs and Boolean functions

- A decision tree can be represented as a **Boolean function**:
- Each path from the root to a leaf codifies a **conjunction** of constraints on the attribute values of instances;
- Different paths that lead to a same classification codify **disjunctions** of conjunctions;
- For each target class we get a specific **DNF (disjunctive normal form) formula**
- **The DT is a set of DNF formulas**



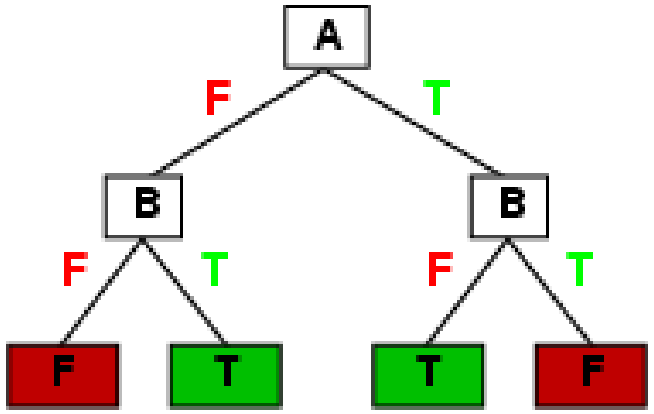
DNF corresponding to YES

(*Outlook*=Sunny ∧ *Humidity*=Normal)
∨
(*Outlook*=Overcast)
∨
(*Outlook* =Rain ∧ *Wind*=Weak)

Expressiveness

- Decision trees can express any function of the input attributes.
- e.g., for Boolean functions, truth table row → path to leaf:

A	B	A xor B
F	F	F
F	T	T
T	F	T
T	T	F



- Trivially, there is a consistent decision tree for any training set with one path to leaf for each example (unless f is nondeterministic in x) but it probably won't generalize to new examples

- Prefer to find **more *compact* decision trees**

Hypothesis spaces

How many distinct decision trees with *n* Boolean attributes?

= number of Boolean functions

= number of distinct truth tables with 2^n rows = 2^{2^n}

- E.g., with 6 Boolean attributes, there are 18,446,744,073,709,551,616 trees

Hypothesis spaces

How many distinct decision trees with n Boolean attributes?

= number of Boolean functions

= number of distinct truth tables with 2^n rows = 2^{2^n}

- E.g., with 6 Boolean attributes, there are 18,446,744,073,709,551,616 trees ($\approx 18{,}4 \times 10^{17}$)

How many purely conjunctive hypotheses (e.g., $Hungry \wedge \neg Rain$)?

- Each attribute can be in (positive), in (negative), or out

  $\Rightarrow 3^n$ distinct conjunctive hypotheses

- More expressive hypothesis space
  - increases chance that target function can be expressed
  - increases number of hypotheses consistent with training set

    $\Rightarrow$ may get worse predictions

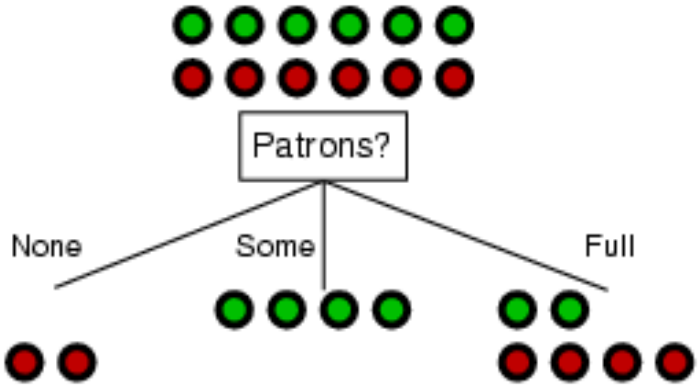
Decision tree learning

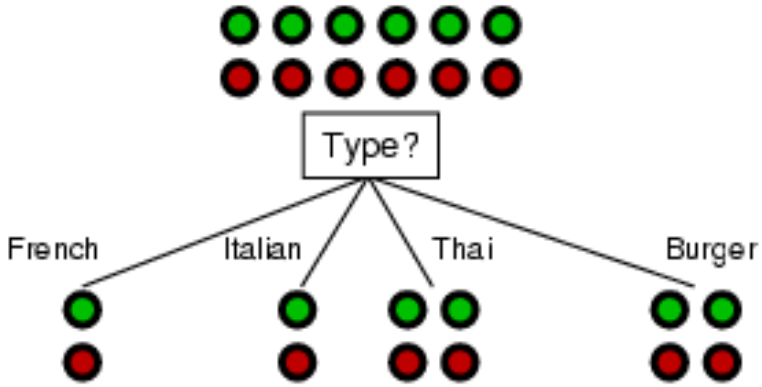
- TARGET: **find a small tree consistent with the training examples**
- IDEA: (recursively) choose "most significant" attribute as root of (sub)tree

```
function DTL(examples, attributes, default) returns a decision tree
    if examples is empty then return default
    else if all examples have the same classification then return the classification
    else if attributes is empty then return MODE(examples)
    else
        best ← CHOOSE-ATTRIBUTE(attributes, examples)
        tree ← a new decision tree with root test best
        for each value v_i of best do
            examples_i ← {elements of examples with best = v_i}
            subtree ← DTL(examples_i, attributes − best, MODE(examples))
            add a branch to tree with label v_i and subtree subtree
        return tree
```

Choosing an attribute

- Idea: a good attribute splits the examples into subsets that are (ideally) "all positive" or "all negative"





- *Patrons?* is a better choice

Using information theory

- To implement `Choose-Attribute` in the DTL algorithm

- Information Content (Entropy):

$$I(P(v_1), \ldots, P(v_n)) = \sum_{i=1} -P(v_i) \log_2 P(v_i)$$

- For a training set containing p positive examples and n negative examples:

$$I(\frac{p}{p+n}, \frac{n}{p+n}) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}$$

Entropy e DTL

- Different DT learning algorithms mainly differ in **how they select the optimal attributes** for the nodes: ID3 (Quinlan, 1983) uses the concepts of Entropy (I) and Information Gain (IG).
- The entropy is a measure of the "*degree of impurity*" of a sample S. Let C be the number of classes and S_c the subset of S of instances in class c, then the entropy is computed with the formula:

$$I(S) = -\sum_{c=1}^{m} p_c \log(p_c), \text{ where } p_c = \frac{|S_c|}{|S|}$$

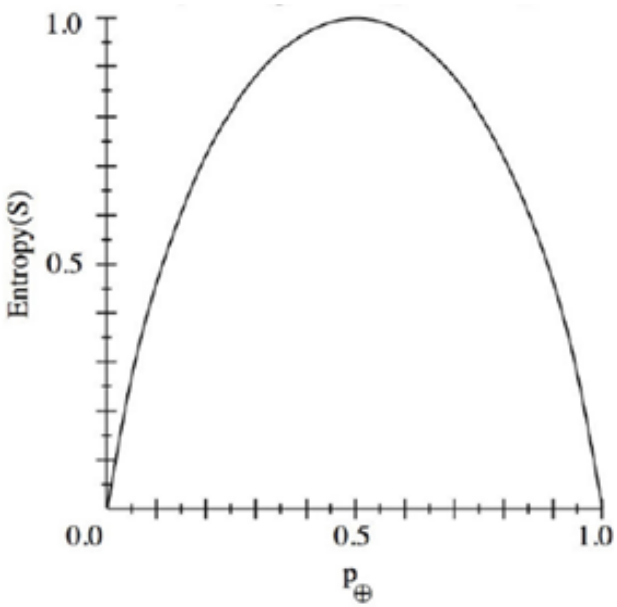
- that, in the case of binary classification, it becomes:

$$I(S) = -p_- \log(p_-) - p_+ \log(p_+)$$

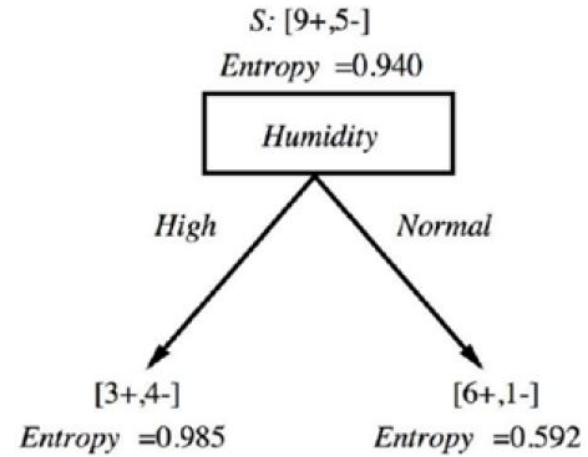
- where p_-, p_+ are the proportion of negative (resp. positive) examples in S.

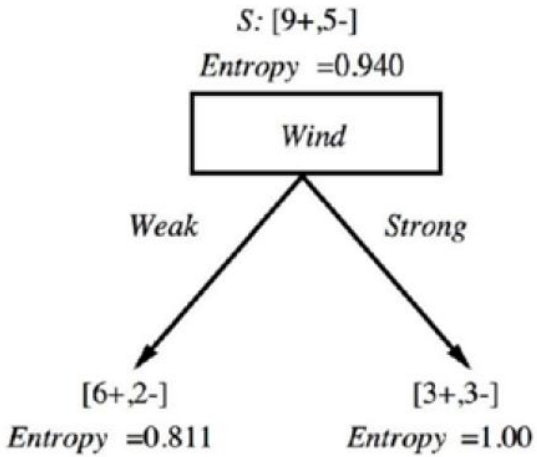
Entropy & Information Gain

Entropy (I)



Information Gain (IG)





Gain (S, Humidity)
= .940 - (7/14).985 - (7/14).592
= .151

Gain (S, Wind)
= .940 - (8/14).811 - (6/14)1.0
= .048

Information gain

- A chosen attribute A divides the training set E into subsets $E_1, \ldots, E_v$ according to their values for A, where A has v distinct values.

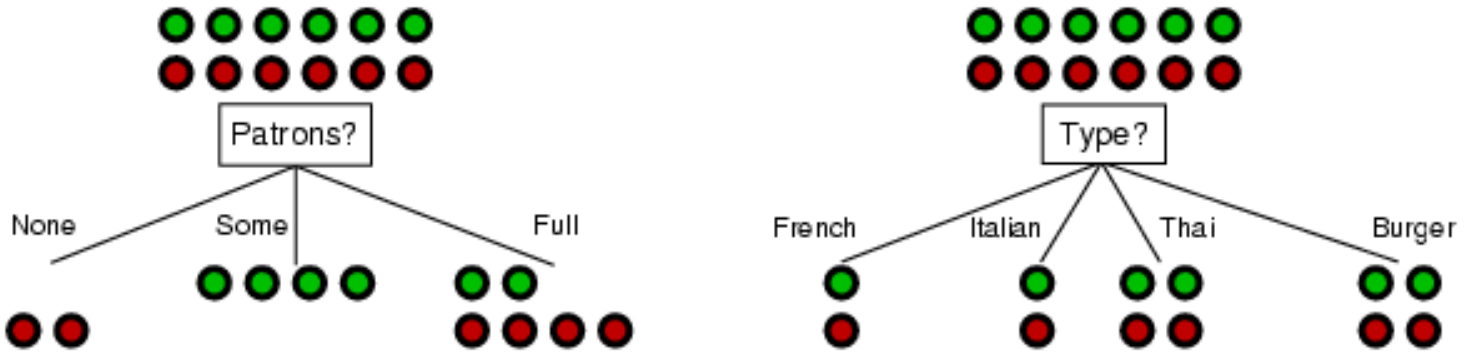
$$remainder(A) = \sum_{i=1}^{v} \frac{p_i + n_i}{p + n} I\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right)$$

- Information Gain (IG) or reduction in entropy from the attribute test:

$$IG(A) = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - remainder(A)$$

- Choose the attribute A with the largest IG

Information gain



For the training set, $p = n = 6$, $I(6/12, 6/12) = 1$ bit

Consider the attributes *Patrons* and *Type* (and others too):

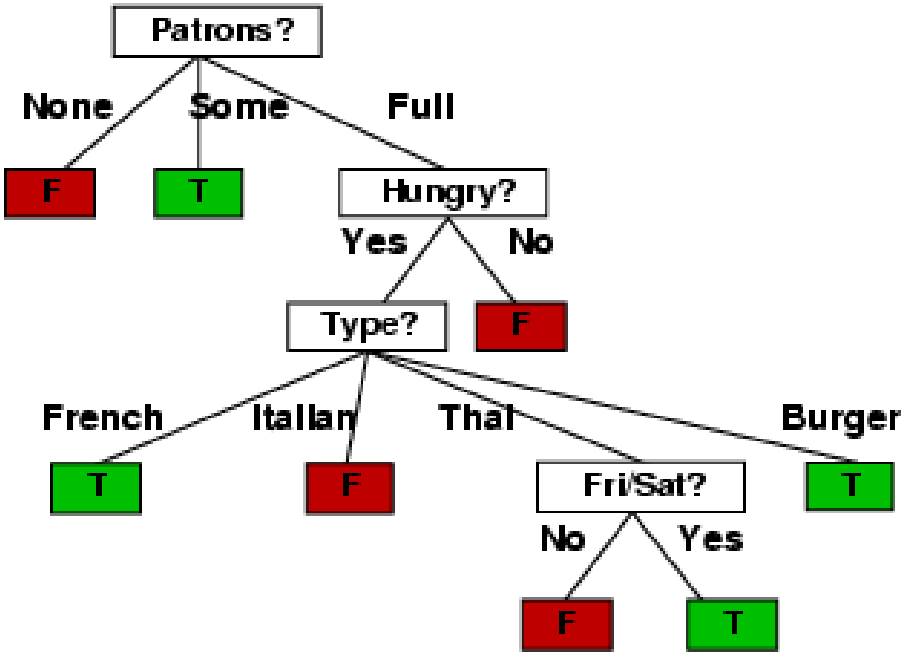
$$IG(Patrons) = 1 - [\frac{2}{12} I(0,1) + \frac{4}{12} I(1,0) + \frac{6}{12} I(\frac{2}{6}, \frac{4}{6})] = .0541 \text{ bits}$$

$$IG(Type) = 1 - [\frac{2}{12} I(\frac{1}{2}, \frac{1}{2}) + \frac{2}{12} I(\frac{1}{2}, \frac{1}{2}) + \frac{4}{12} I(\frac{2}{4}, \frac{2}{4}) + \frac{4}{12} I(\frac{2}{4}, \frac{2}{4})] = 0 \text{ bits}$$

Patrons has the highest IG of all attributes and so is chosen by the DTL algorithm as the root

Example contd.

- Decision tree learned from the 12 examples:



- Substantially simpler than "true" tree---a more complex hypothesis isn't justified by small amount of data

Esercizi

- Usare l'algoritmo DT learning per calcolare manualmente il decision tree che corrisponda ad una certa semplice formula booleana mediante la applicazione di operazioni logiche (AND, OR, XOR, ...) in n variabili.
  - Suggerimento: Considerare la tabella di verità di una formula come il training set di partenza

- Selezionare la affermazione corretta tra le seguenti:
  - Nel DT learning lo splitting si basa sull'attributo con l'IG minimo
  - Il DT learning visto a lezione produce sempre il DT con la profondità minima
  - L'intervallo dell'IG di un attributo (su un qualsiasi training set) è sempre [0,1]
  - L'Information Gain ha una forte polarizzazione su attributi che hanno molti valori (test booleani con branching factor molto grande, ad es. le date individuali)

-
  -
  -
  -
  -

Il controllo dell'apprendimento

Come gestire il training set?

Quali sono le evidenze dell'errore?

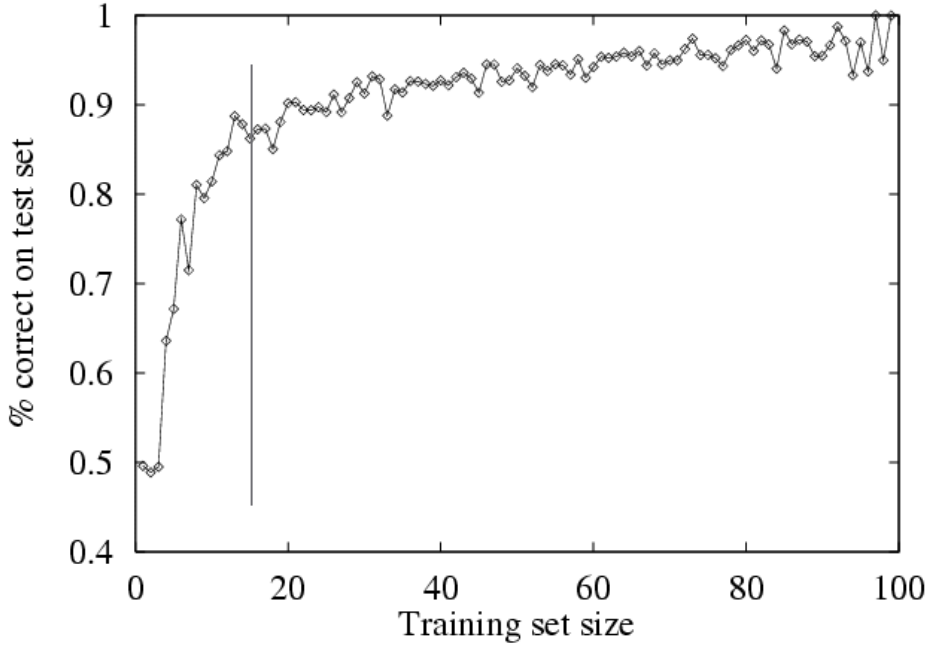
Come migliorare il mio modello a fronte di errori?

Quando debbo smettere di apprendere?

Performance measurement

- How do we know that $h \approx f$?
  1. Use theorems of computational/statistical learning theory
  2. Try h on a new test set of examples
     (use same distribution over example space as training set)

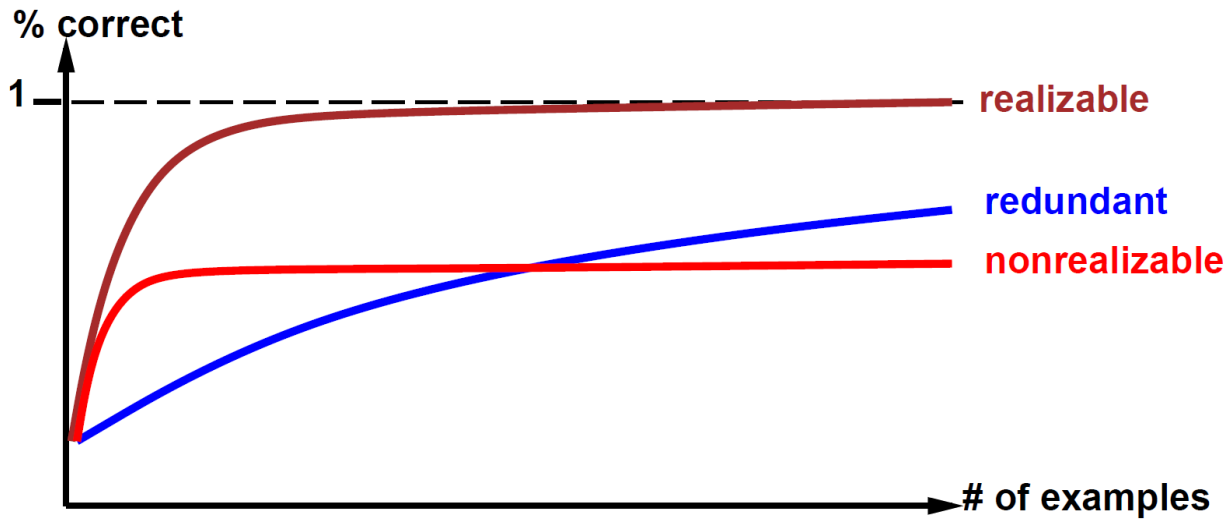
Learning curve = % correct on test set as a function of training set size



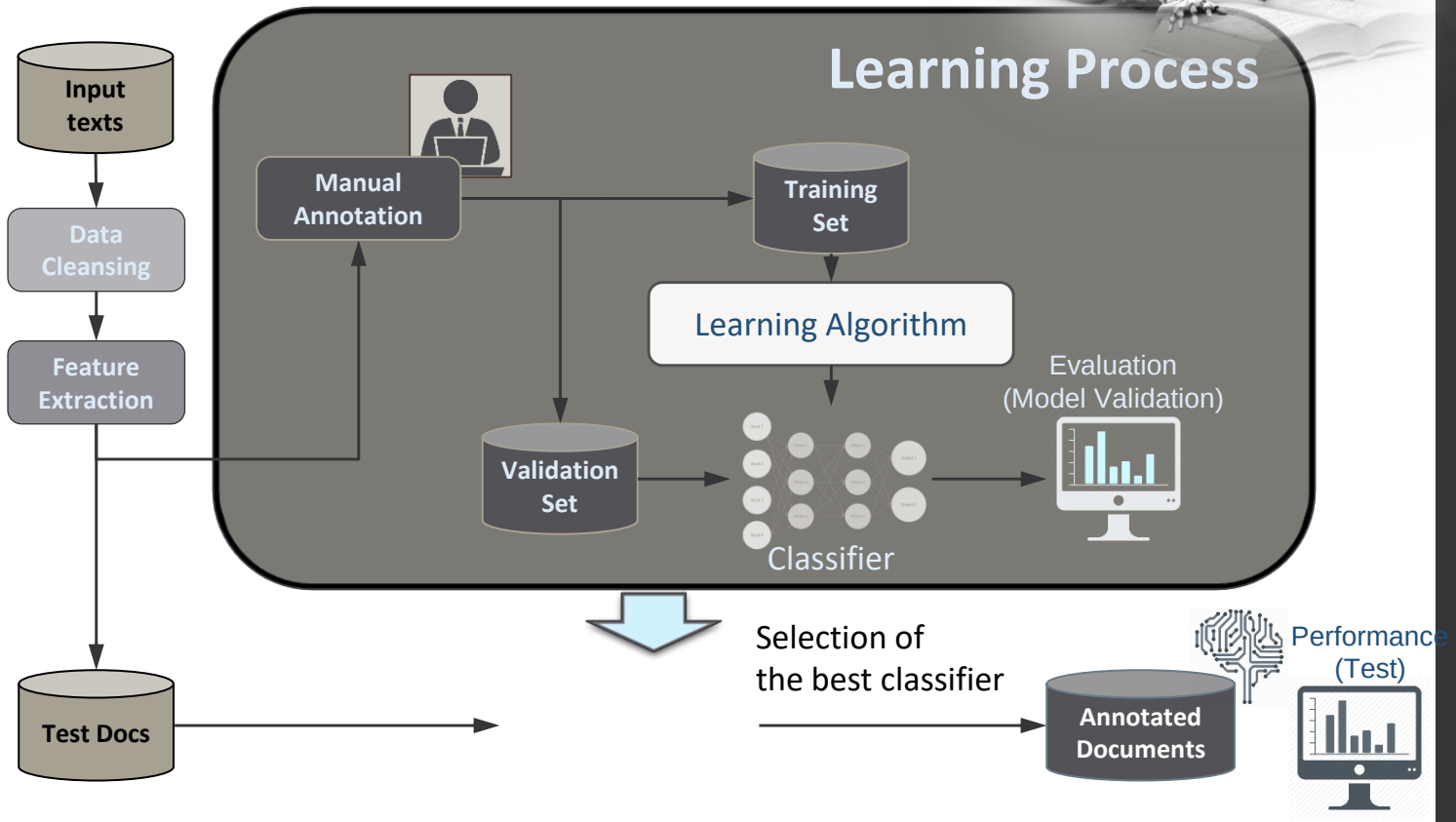
Performance measurements (2)

- **Learnability** depends on
  - **realizable** kind of performances vs.

    … **non-realizable** ones
  - **Non-realizability** depends on
    - Missing attributes
    - Limitation on the hypothesis space (e.g. non expressive functions)
  - **Redundant expressiveness** is related to cases where a a largenumber of irrelevant attributes are used



Machine Learning workflow



Il controllo dell'apprendimento

Come gestire il training set?

Quali sono le evidenze dell'errore?

Come migliorare il mio modello a fronte di errori?

Quando debbo smettere di apprendere?

Evaluation of a ML system

- Performance Evaluation Metrics
  - Evaluation Metrics for Classifiers
- Parameter Tuning and Evaluation Methods

Classifier Evaluation: Confusion Matrix

		PREDICTED VALUE		
ACTUAL VALUE		Class A	Class B	Class C
	Class A	38	12	0
	Class B	5	43	2
	Class C	6	0	44

$$accuracy = \frac{\#correct\ classifications}{\#classifications} = \frac{38 + 43 + 44}{150} = 83.33\%$$

$$error\ rate = \frac{\#incorrect\ classifications}{\#classifications} = \frac{12 + 5 + 2 + 6}{150} = 16.67\%$$

Evaluation with skewed data

- Accuracy is not a suitable metric for task with imbalanced classes (for instance a spam detector)

		PREDICTED VALUE	
ACTUAL VALUE		Spam	Non-Spam
	Spam	0	10
	Non-Spam	0	9990

Very bad performance on the Spam class, that is the target of the classifier!! … nonetheless …

$$accuracy = \frac{\#correct\ classifications}{\#classifications} = \frac{9990}{10000} = 99.9\%$$

Single Class Metrics

		PREDICTED VALUE	
		Class C	**Not Class C**
ACTUAL VALUE	**Class C**	**TP** True Positive	**FN** False Negative
	Not Class C	**FP** False Positive	**TN** True Negative

$$precision = \frac{TP}{TP + FP}$$

what percentage of instances the classifier labeled as positive are actually positive?

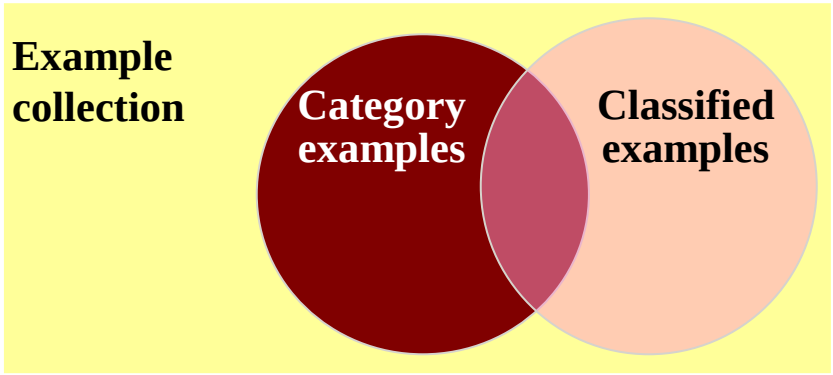
$$recall = \frac{TP}{TP + FN}$$

what percentage of positive instances did the classifier label as positive?

$$F1 = \frac{2 \times precision \times recall}{precision + recall}$$

F-measure is the harmonic mean of precision and recall

Class-based evaluation



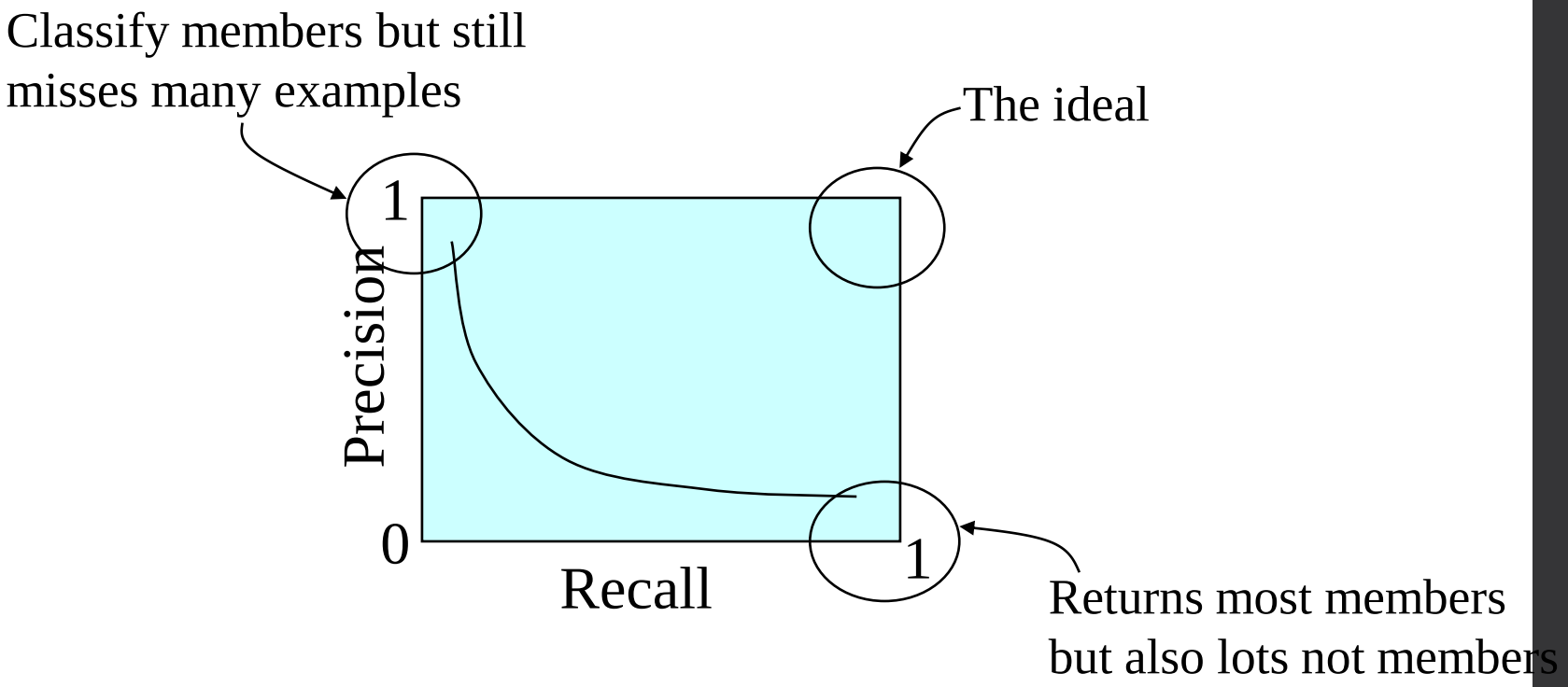
	Classified	Rejected
Not Members	Classified but not Members	Rejected & not Members
Members	Classified & Members	Rejected but Members

$$precision = \frac{\#\ of\ Members\ Classified}{\#\ of\ Members\ Classified\ +\ \#\ of\ Classified\ not\ Members}$$

$$recall = \frac{\#\ of\ Members\ Classified}{\#\ of\ Members\ Classified\ +\ \#\ of\ Rejected\ Members}$$

What about accuracy???

Trade-off between Precision and Recall



Other class based measures

Precision and Recall of C_i

- a, corrects (TP_i)
- b, mistakes (FP_i)
- c, instances of a $Class_i$ that are not actually retrieved, (FN_i)

The *Precision* and *Recall* are defined by the above counts:

$$Precision_i = \frac{a_i}{a_i + b_i}$$

$$Recall_i = \frac{a_i}{a_i + c_i}$$

		PREDICTED VALUE		
ACTUAL VALUE		Class A	Class B	Class C
	Class A	38	12	0
	Class B	5	43	2
	Class C	6	0	44

- $\text{Precision}_A = 38/(38+5+6)=38/49$
- $\text{Recall}_A = 38/(38+12)=38/50$

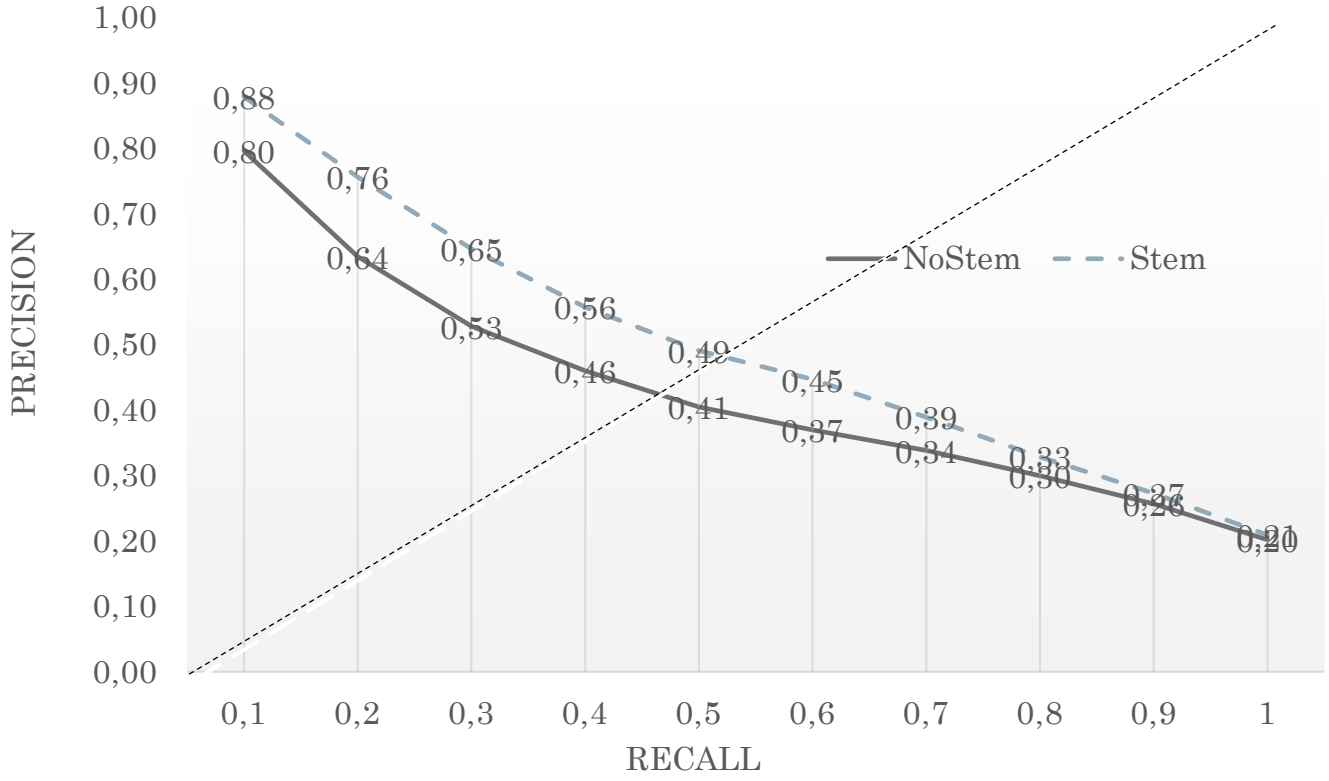
- $\text{Precision}_B = 43/(43+12)=43/55$
- $\text{Recall}_C = 44/(44+6)=44/50$

Performance Measurements (cont'd)

- Breakeven Point
  - Find thresholds for which
    Recall = Precision
  - Interpolation
- F-measure $F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$
  - Harmonic mean between precision and recall
- Global performance on more than two categories
  - Micro-average
    - The counts refer to classifiers
  - Macro-average (average measures over all categories)

Break-even Point

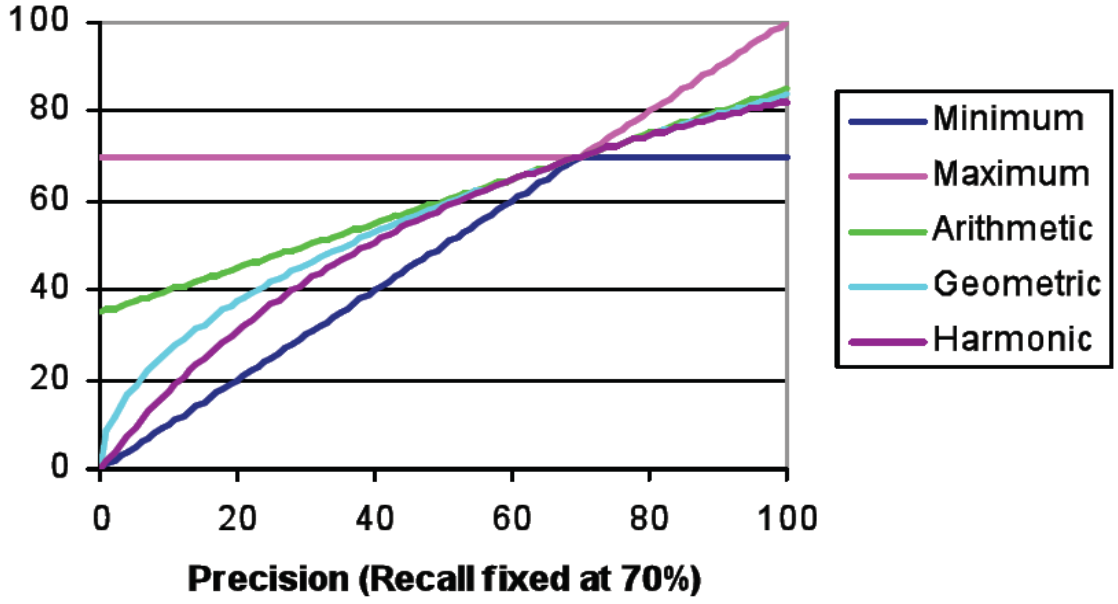
- The BEP is the interpolated estimate of the value for which Recall=Precision



- It shows the superiority of methods whose behavior is closer to the (1,1) ideal performance

Averaging Precision & Recall: A comparison

$$F_1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$



$\min(p, r)$
$\max(p, r)$

$$arithM(p, r) = \frac{p + r}{2}$$

$$geomM(p, r) = \sqrt{p \cdot r}$$

$$harmM(p, r) = \frac{2}{p^{-1} + r^{-1}}$$

Averaging Precision & Recall: cross-categorical analysis

- Individual scores characterize the performance about each specific class
- Simple macro averaging can be applied to have

$$MPrecision = \frac{1}{n}\sum_{i=1}^{n} Precision_i$$

$$MRecall = \frac{1}{n}\sum_{i=1}^{n} Recall_i$$

$$MF_1 = \frac{2 \cdot MPrecision \cdot MRecall}{MPrecision + MRecall}$$

F-measure e MicroAverages

$$F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$\mu Precision = \frac{\sum_{i=1}^{n} a_i}{\sum_{i=1}^{n} a_i + b_i}$$

$$\mu Recall = \frac{\sum_{i=1}^{n} a_i}{\sum_{i=1}^{n} a_i + c_i}$$

$$\mu BEP = \frac{\mu Precision + \mu Recall}{2}$$

$$\mu f_1 = \frac{2 \times \mu Precision \times \mu Recall}{\mu Precision + \mu Recall}$$

		PREDICTED VALUE		
ACTUAL VALUE		Class A	Class B	Class C
	Class A	38	12	0
	Class B	5	43	2
	Class C	6	0	44

- $\text{Precision}_A = 38/(38+5+6)=38/49$
- $\text{Precision}_B = 43/(43+12)=43/55$
- $\text{Precision}_C = 44/(44+2)=44/46$
- Segue che:

$$\text{Mprecision}=1/3(38/49 + 43/55 + 44/46)$$

		PREDICTED VALUE		
ACTUAL VALUE		Class A	Class B	Class C
	Class A	38	12	0
	Class B	5	43	2
	Class C	6	0	44

- $\text{Precision}_A = 38/(38+5+6)=38/49$
- $\text{Precision}_B = 43/(43+12)=43/55$
- $\text{Precision}_C = 44/(44+2)=44/46$
- Segue che:

$$\mu\text{Precision}=(38+43+44)/(38+43+44+11+12+2)$$

Il controllo dell'apprendimento

Come gestire il training set?

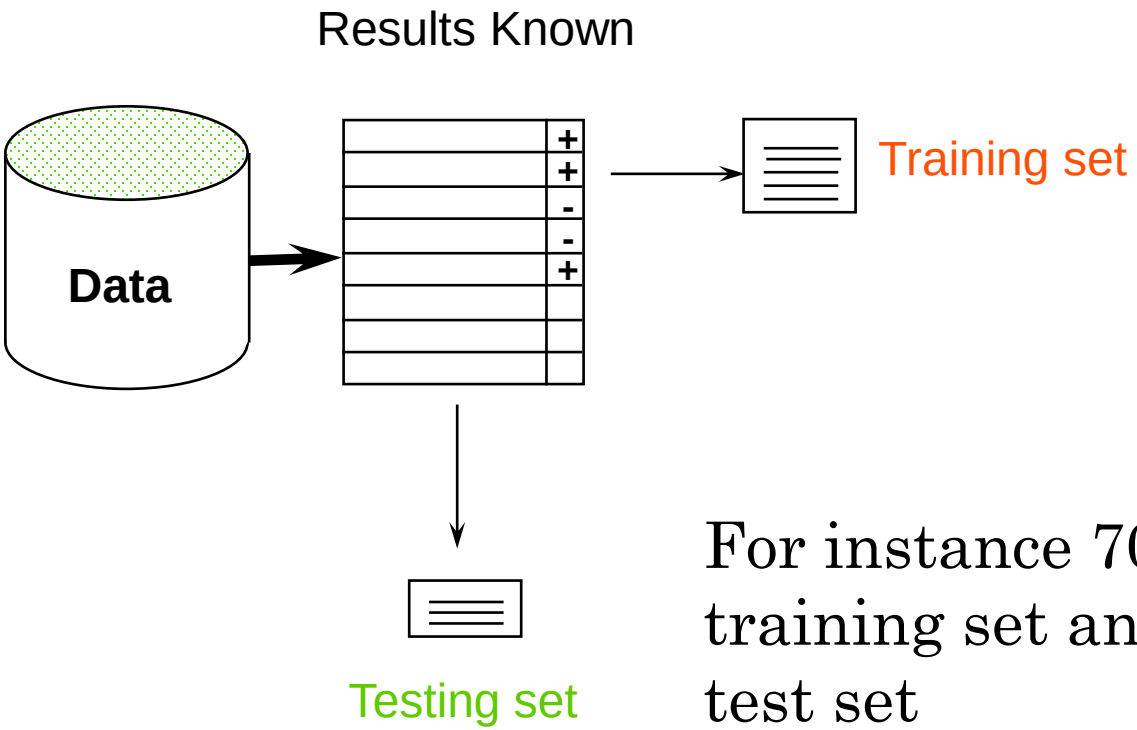
Quali sono le evidenze dell'errore? Come misurarle?

Come migliorare il mio modello a fronte di errori? Quando debbo smettere di apprendere?

Testing Data

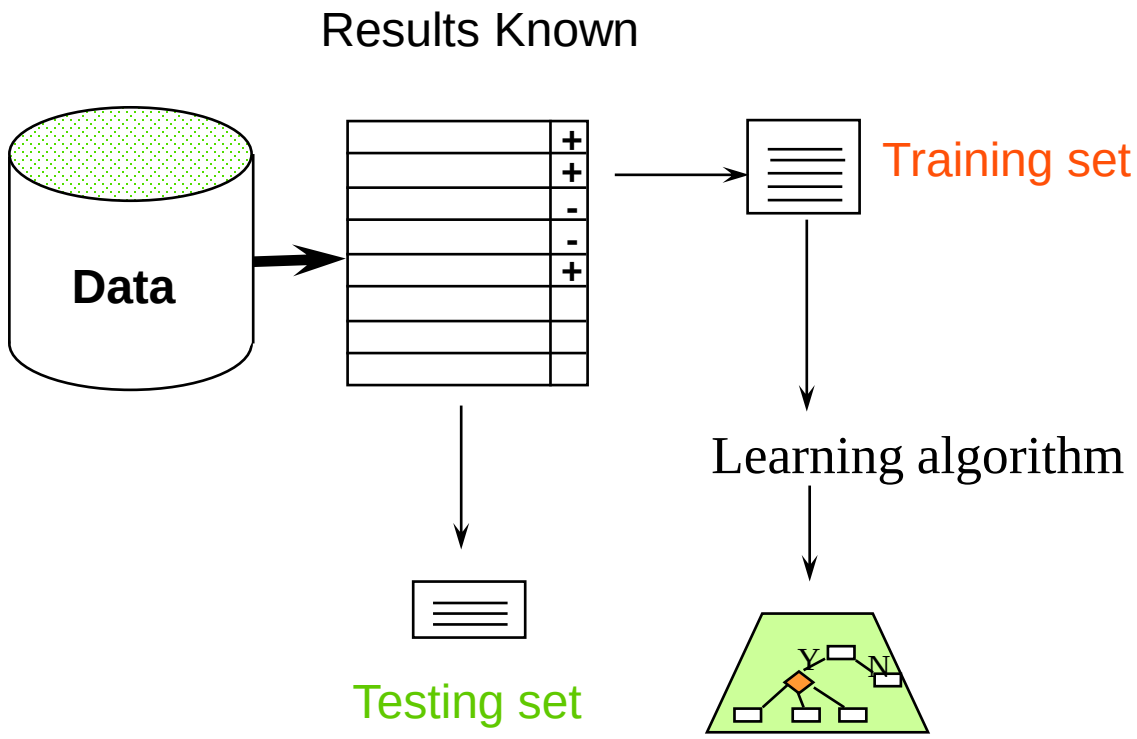
- To obtain a reliable estimation, test data **must be** instances **not** employed for the training step:
  - Error on the training data is *not* a good indicator of performance on future data, because new data will probably not be **exactly** the same as the training data!
  - *Overfitting* – fitting the training data too precisely - usually leads to poor results on new data
  - We want to evaluate how much accurate predictions of the model we learned are, and not other computational aspects (e.g. its memorization capability)

Step 1: dataset splitting

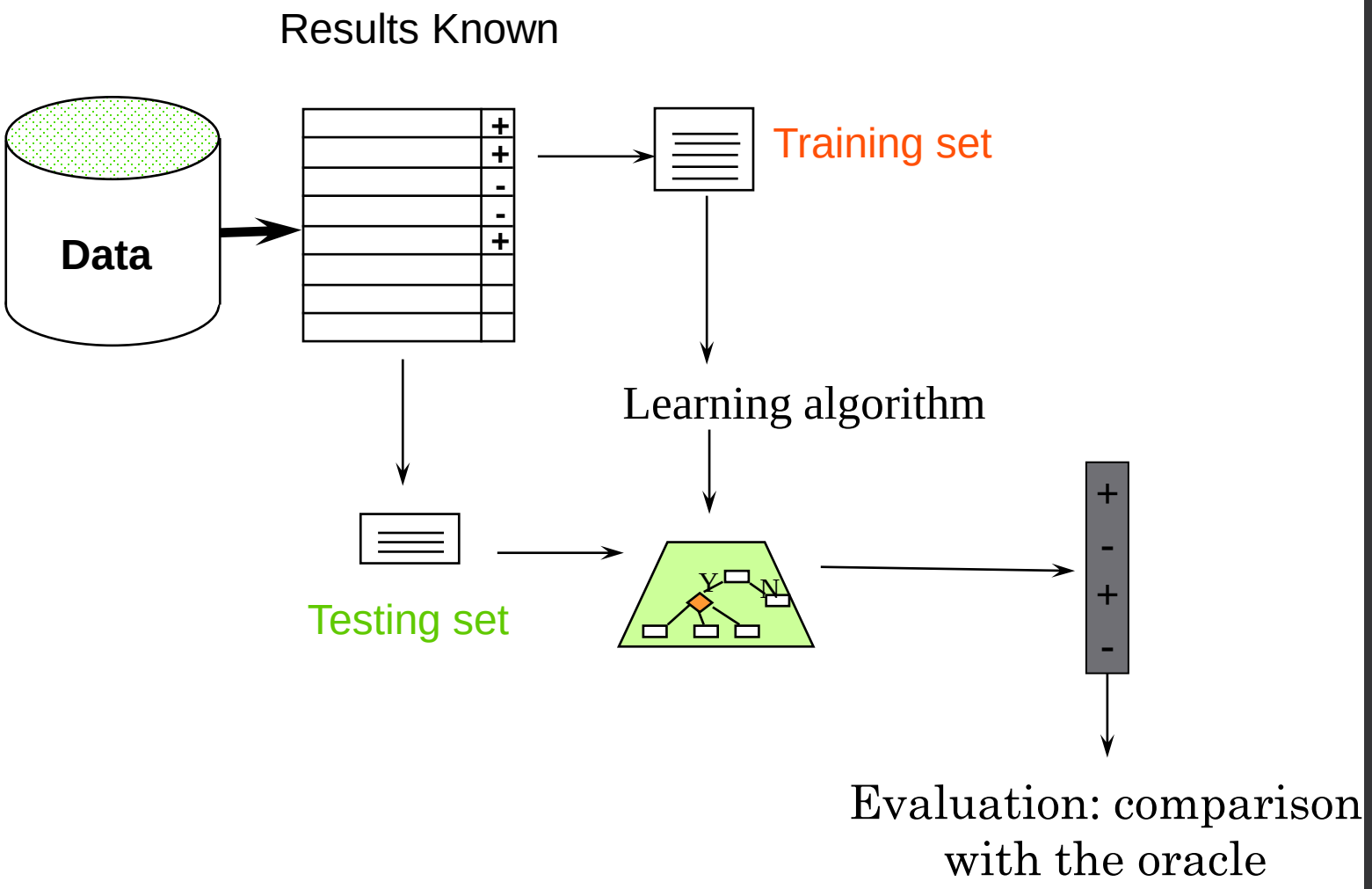


For instance 70% in the training set and 30% in the test set

Step 2: learning phase



Step 3: testing the model

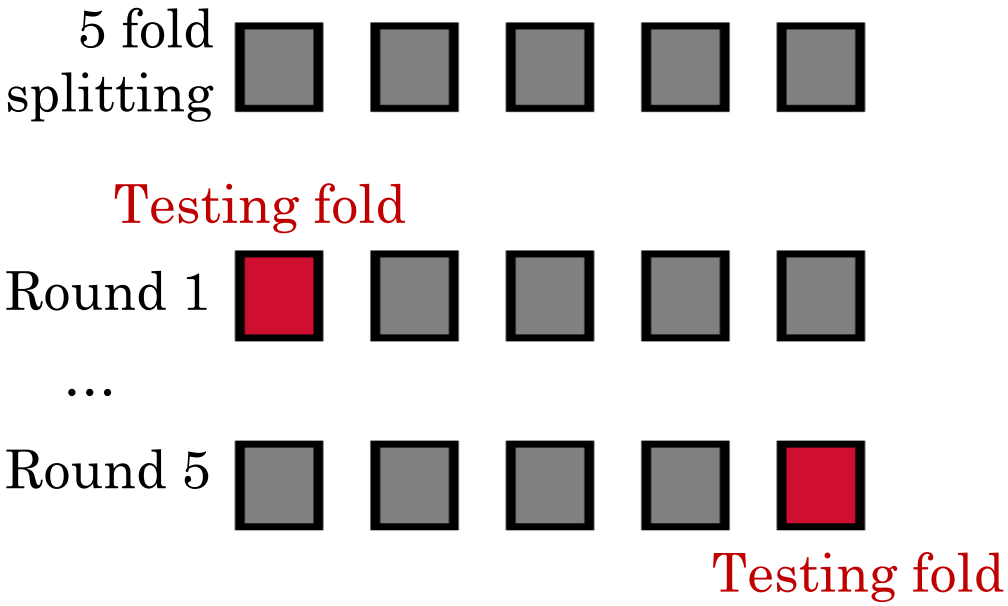


Evaluation on Few Data
Poor Training Conditions

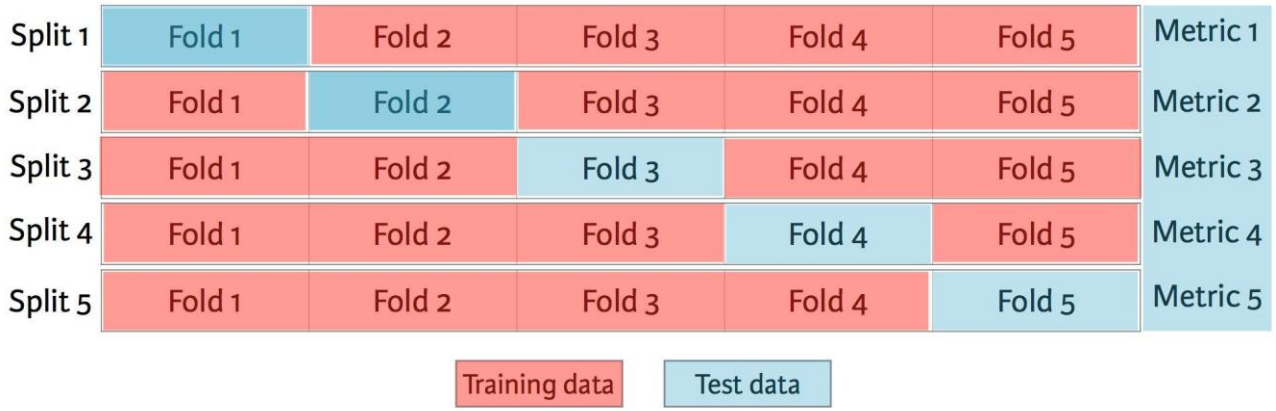
- PROBLEM: When data is scarce (totally or for a single class), a single evaluation process could not be enough representative
  - The testing set could contain too few instances to produce a reliable result
- SOLUTION: SAMPLING!
  - The evaluation process must be repeated with different splitting

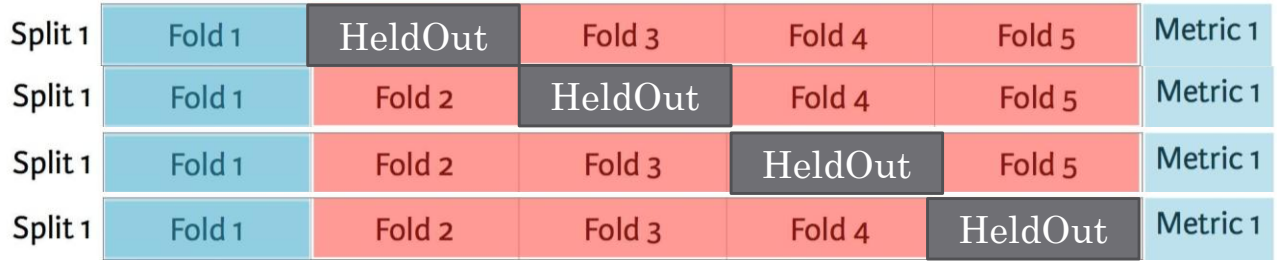
N-Fold Cross Validation

- Data is split into *n* subsets of equal size
- Each subset in turn is used for testing and the remainders *n-1* for training
- The metrics estimated in each round are averaged



Cross-validation and the Hold-out





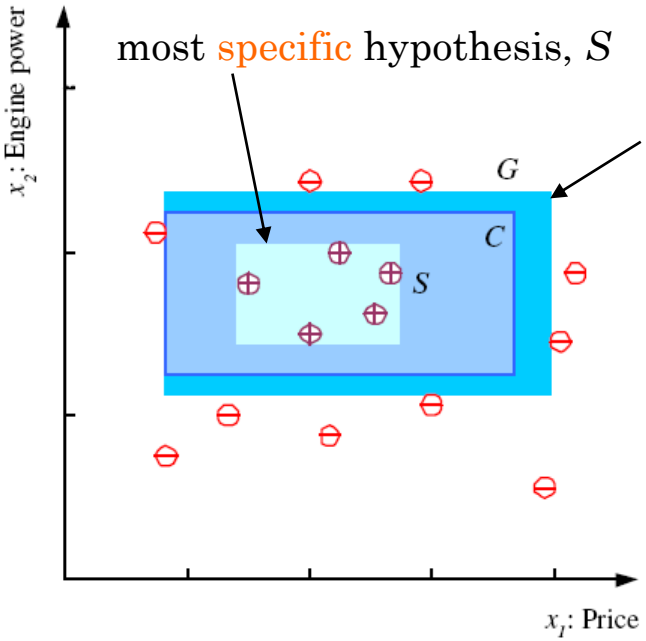
Tuning a Classifier

- Most of ML algorithms depends on some parameters
  - Examples: k in KNN, w_i in Rocchio, $p(w_i \mid c_j)$ for NB
- The best configuration must be choosen after a proper tuning stage:
  - A set of configurations must be established (for instance, k=1,2,5,10,…,50)
  - Each configuration must be evaluated on a validation (or tuning) set

… short look at
model selection

ning © The

(Vector) Spaces, Functions and Learning



$$h \in \mathcal{H}$$

The $h \in \mathcal{H}$ floats between S and G to be consistent

It makes up the version space

(Mitchell, 1997)

Model selection

- We try to find the model with the best balance of complexity and the fit to the training data
- Ideally, we would select a model from a nested sequence of models of increasing complexity (VC-dimension)

  Model 1 d_1
  Model 2 d_2
  Model 3 d_3

  where $d_1 \leq d_2 \leq d_3 \leq \ldots$

- The model selection criterion is: find the model class that achieves the lowest upper *bound* on the expected loss

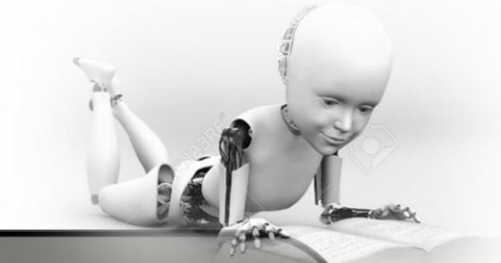
  Expected error $\leq$ Training error $+$ Complexity penalty

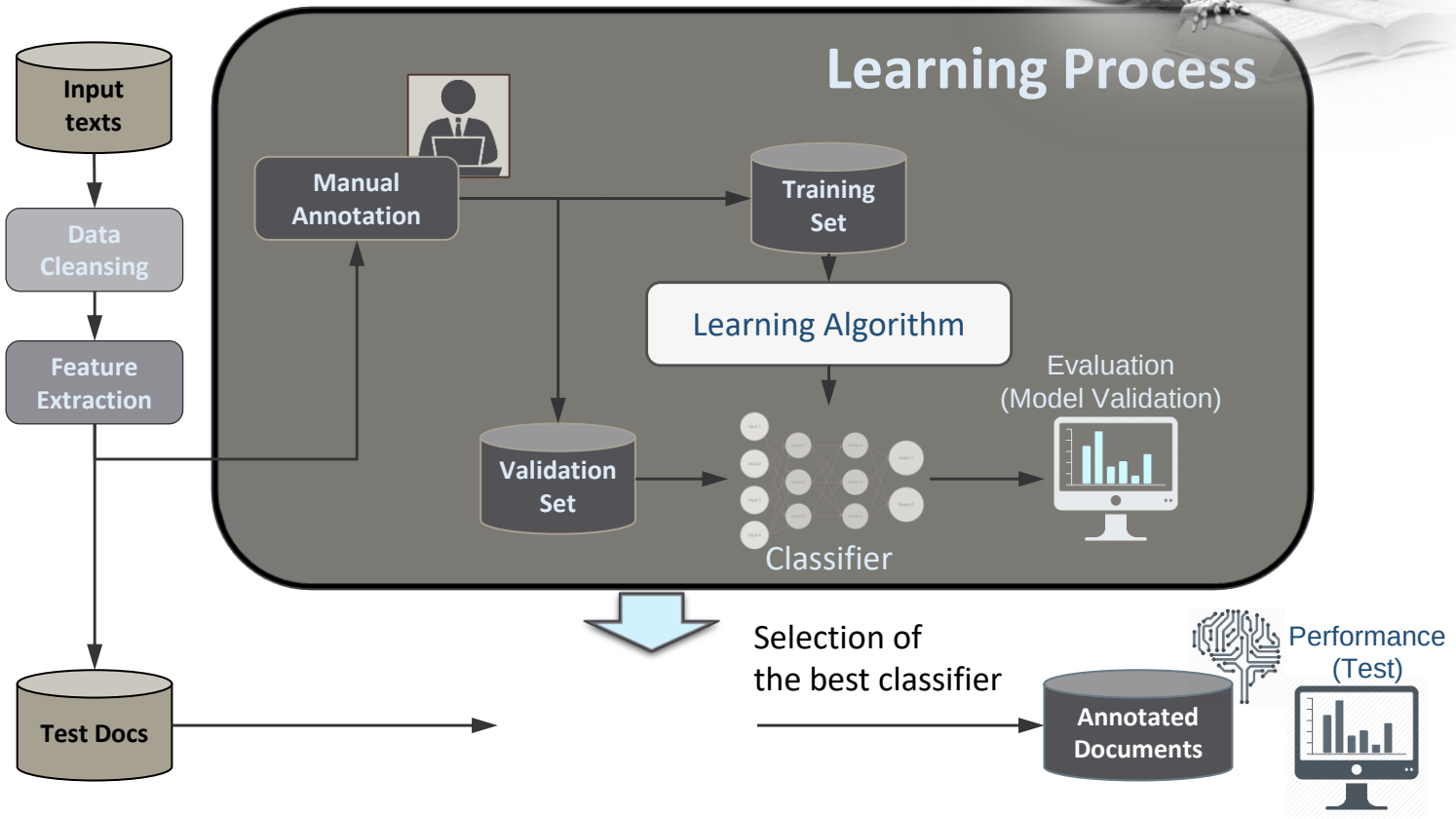
Alternatives to VC-dim-based model selection

- What could we do instead of the scheme below?
  - Cross-validation

i	f_i	TRAINERR	10-FOLD-CV-ERR	Choice
1	f_1			
2	f_2			
3	f_3			?
4	f_4			
5	f_5			
6	f_6			

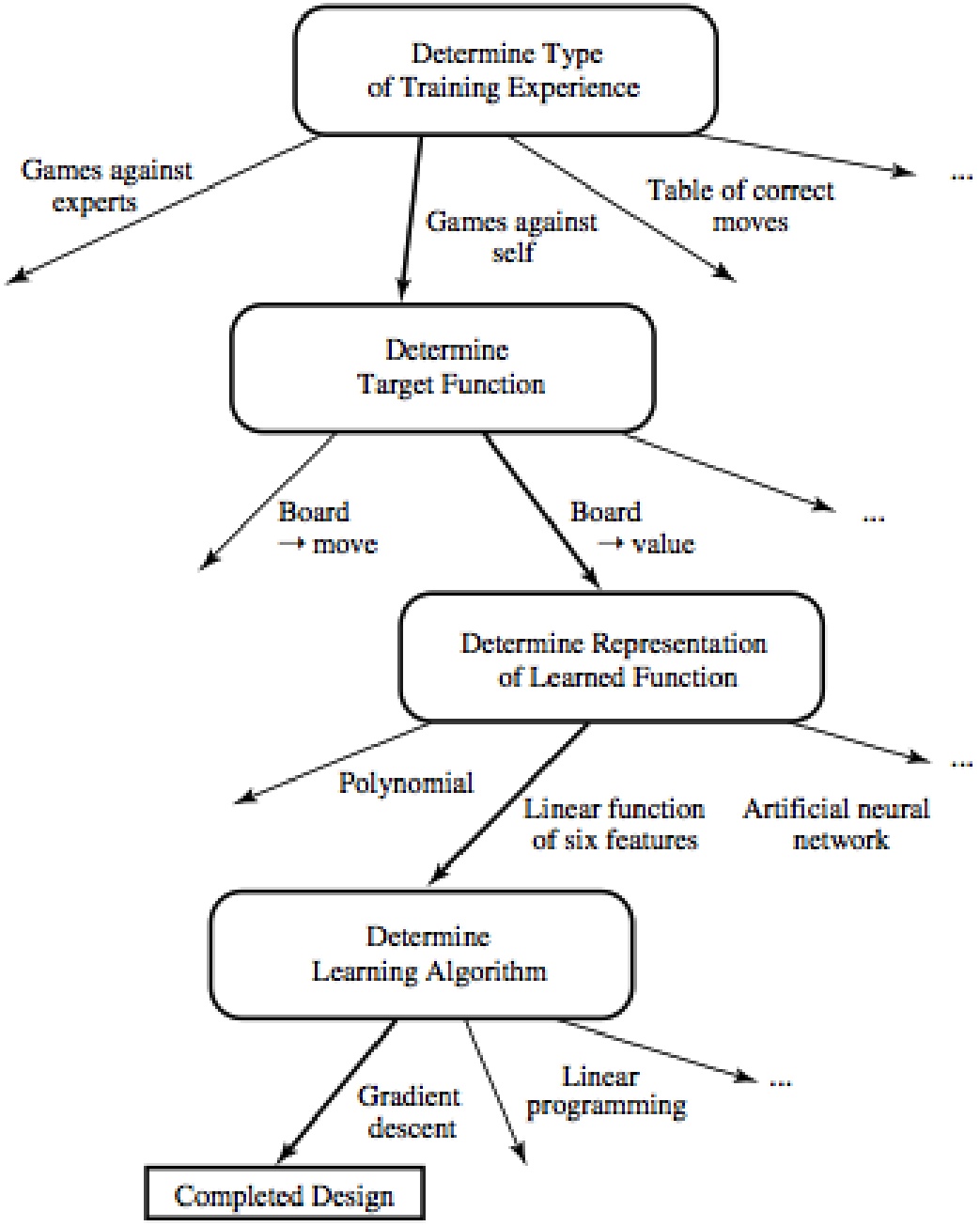
Machine Learning workflow





Design of a learning system

Mitchell, 1997

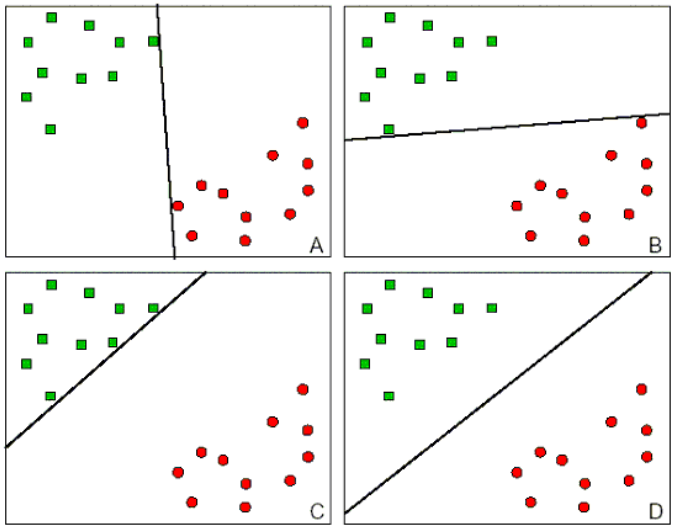


Machine Learning Tasks

- Supervised learning da esempi
  - Classification
    - Approcci dicriminativi
    - Approcci generative
    - Outlier and deviation detection
  - Regression
  - Dependency modeling
    - Discovery di Associazioni/Relazioni, Sommari, Inferenza/Causalità
  - Sequence Classification
    - Temporal learning
    - Trend analysis and change/anomaly detection
- Unsupervised learning
  - Clustering
  - Embedding ottimo: Enconding/Decoding
    - Representation Learning for Images
    - PreTraining as optimal encoding

Metodi di ML: selezione dei modelli

- Approcci discriminativi
  - Lineari
  - $h(x) = sign(\boldsymbol{W} \cdot \boldsymbol{x} + b)$



  - Approcci probabilisti
    - Stima delle probabilità $p(\mathcal{C}_k|\mathbf{x})$ attraverso un training set
    - Modello generativo ed uso della inversione Bayesiana

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})}.$$

Riferimenti Bibliografici

- *AIMA*, Chapter 18, 18.1-18.4, 18.6-18.7, 18.11
- READING. *Machine Learning*, Tom Mitchell, Mc Graw-Hill International Editions, 1997 (Cap 3).
- …
- L'Algoritmo Definitivo, Pedro Domingos, Bollati Boringhieri, 2016

